



Introduction to Blockchain

CSE-355

Ahmed Akhtar

Department of Computer Science

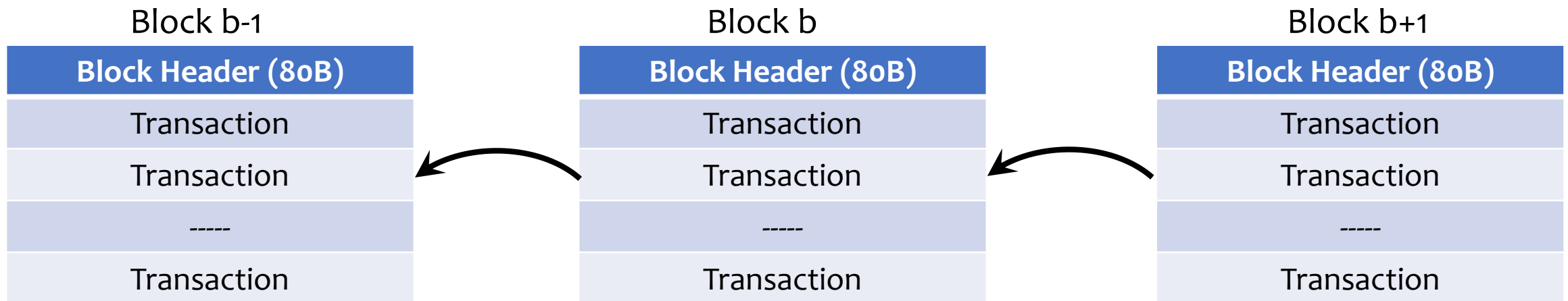
SMCS, IBA Karachi

Spring 2026

Lecture 13

Tuesday, Mar 3, 2026

Blockchain Block Header



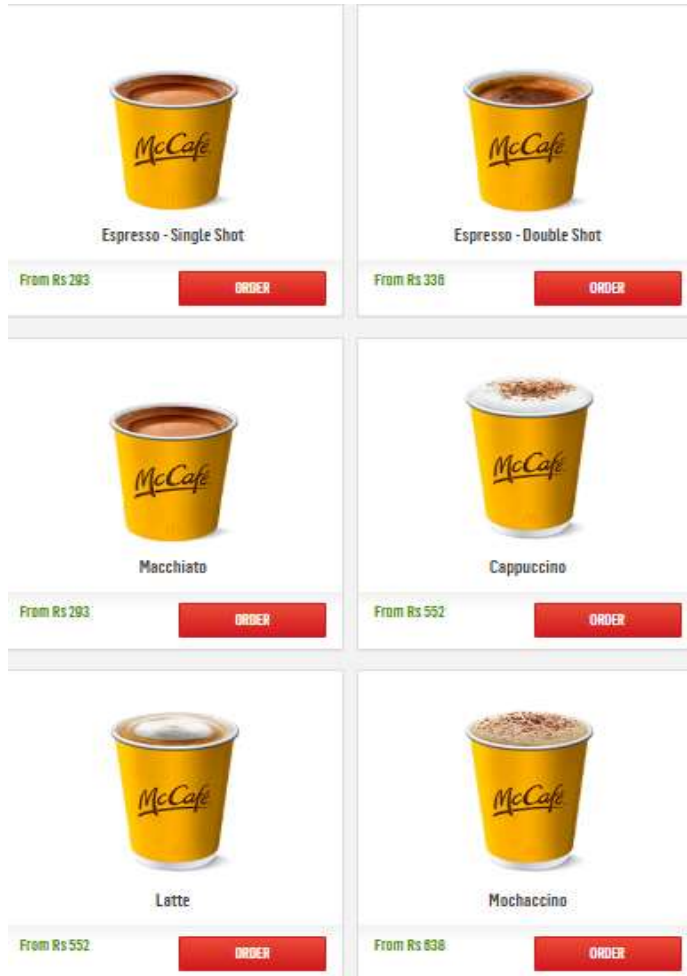
There are 6 fields in the block header:

1. Version (4B) – for exchanging messages and signaling the protocol rules
2. Previous block hash (32B) – for linking current block to its parent block
3. Merkle root hash (32B) – for summarizing & maintaining integrity of transactions in the block
4. Timestamp (4B) – for recording block creation time; up to 2 hour variation is acceptable because Bitcoin nodes are not concerned about timing synchronization
5. Target_Hash (4B) – for defining the difficulty of finding the block header hash; adjusts every 2016 blocks (~ two weeks) to ensure 10 minute average block generation time
6. Nonce (4B) – for iterating values during hash attempts to find a valid block header

$\text{Block_Header} = [\text{Version} + \text{PreviousBlockHash} + \text{MerkelRootHash} + \text{Timestamp} + \text{DifficultyTarget} + \text{Nonce}]$

$\text{Block_Header_Hash} = \text{SHA256}(\text{SHA256}(\text{Block_Header}))$

Can We Pay for Coffee in BTC?



- Bitcoin price are too high for a single cup of coffee
- Average block generation time is 10 minutes, so do we want to wait this long every time for a coffee payment to go through?
- Do we really want every cup of coffee we drink broadcast to the whole world?
- Billions of coffee cups are consumed every day so can bitcoin process all these coffee transactions?

Bitcoin Statistics: Jan 22, 2026

- Bitcoins in circulation: **19.98 Million** (over \$2T market cap)
- Total will be **21 Million** (last will be mined about in the year **2140**)
- Bitcoins left to be mined: **1.02 Million** (Total issued: **95.1%**)
- Blockchain Size: **715 GB**
- New Bitcoin Blocks per day (presently): **144** (on average)
- New Bitcoins per day (presently): **450** ($144 \times \underline{\mathbf{3.125}}$)
- About **4 Million** Bitcoins were lost forever (estimate)
- Transactions per second: **7** (VISA has about **24k**)
- Bitcoin Price: **\$ 66k**

Does Bitcoin look like a peer-to-peer electronic cash system?

Bitcoin – Decentralized Digital Currency

Bitcoin: A Peer-to-Peer Electronic Cash System

Satoshi Nakamoto
satoshiin@gmx.com
www.bitcoin.org

Abstract. A purely peer-to-peer version of electronic cash would allow online payments to be sent directly from one party to another without going through a financial institution. Digital signatures provide part of the solution, but the main benefits are lost if a trusted third party is still required to prevent double-spending. We propose a solution to the double-spending problem using a peer-to-peer network. The network timestamps transactions by hashing them into an ongoing chain of hash-based proof-of-work, forming a record that cannot be changed without redoing the proof-of-work. The longest chain not only serves as proof of the sequence of events witnessed, but proof that it came from the largest pool of CPU power. As long as a majority of CPU power is controlled by nodes that are not cooperating to attack the network, they'll generate the longest chain and outpace attackers. The network itself requires minimal structure. Messages are broadcast on a best effort basis, and nodes can leave and rejoin the network at will, accepting the longest proof-of-work chain as proof of what happened while they were gone.

Satoshi envisioned Bitcoin as a peer-to-peer digital currency for all kind of online payments



<https://news.bitcoin.com/10-years-ago-bitcoins-genesis-block-changed-the-course-of-history/>

Bitcoin as a Micropayment Platform?

Is there a way to make bitcoin a micropayment platform without sacrificing the decentralization or security?

Bitcoin Scripts Revisited

What Can Bitcoin Opcodes Do?

- **Check signatures**
Opcodes like OP_CHECKSIG verify that a transaction is authorized by the owner of a private key –
- **Combine conditions**
Opcodes like OP_IF, OP_ELSE, and OP_ENDIF allow multiple spending conditions (e.g., *Alice OR Bob can spend*)
- **Require multiple approvals**
Opcodes like OP_CHECKMULTISIG allow *M-of-N* signatures (e.g., 2 out of 3 people must approve)
- **Lock funds by time**
Opcodes like OP_CHECKLOCKTIMEVERIFY (CLTV) can prevent spending until a certain date or block height –
- **Lock funds by delay**
Opcodes like OP_CHECKSEQUENCEVERIFY (CSV) require waiting some time *after* a transaction is created

What is possible with Bitcoin Scripts?

Lock BTC with different spending conditions inside Bitcoin transactions

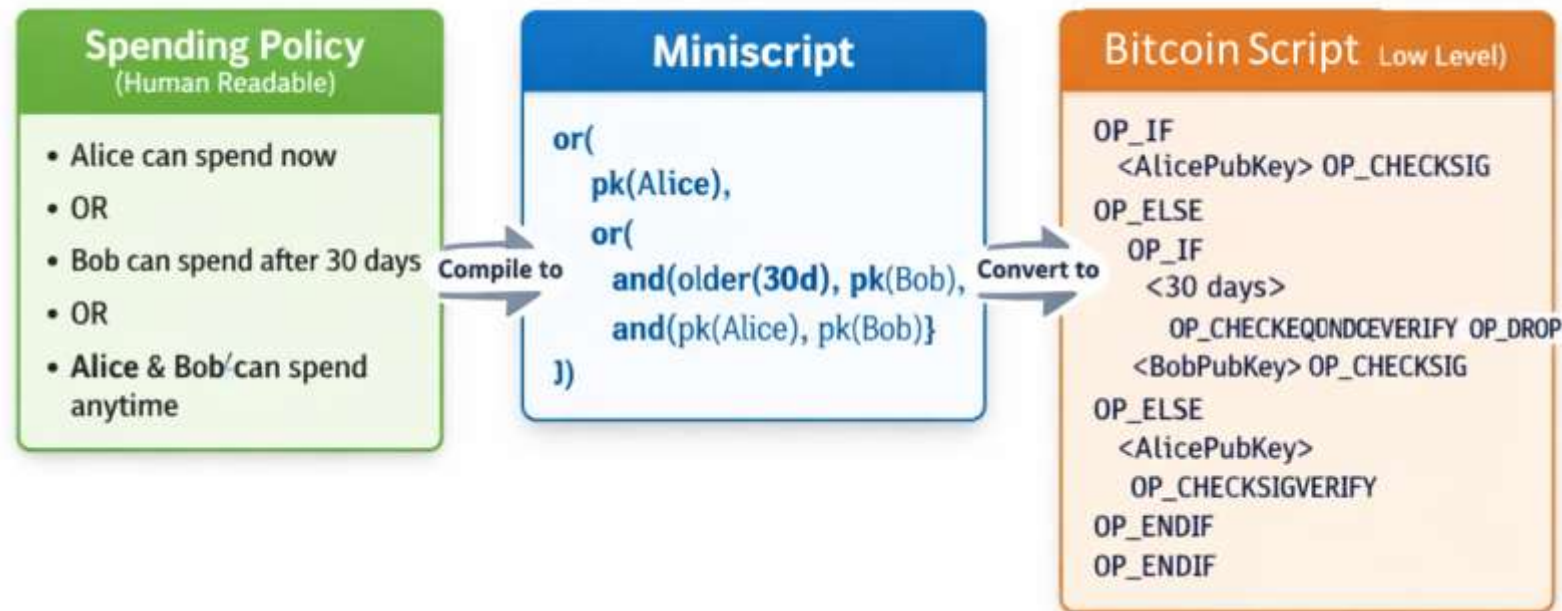


- Time locks
 - Spending via absolute-time-based lock (nLockTime & CheckLockTimeVerify (CLTV))
 - Spending via relative-time-based lock (nSequence & CheckSequenceVerify (CSV))
- Hash locks
 - Spending via Hash locks
 - Spending via Hashed Time lock Contract (HTLC)

But scripts are low-level and hard to write
Bitcoin Script is stack-based and not designed to be human-friendly

Miniscript

- Miniscript is like **TypeScript** for Bitcoin
- Just as TypeScript turns into JavaScript so browsers can read it, Miniscript turns into **Bitcoin Script** so the blockchain can read it
- It allows developers to build "Smart Contracts" on Bitcoin without needing a PhD in Opcodes



Comparison

RAW Bitcoin Script

- Hard to read
- Easy to mess up
- Not intuitive
- Hard to verify

Miniscript

- Structured
- Readable
- Matches the policy
- Easier to reason about

Bitcoin Transactions – UTXO Model

- Bitcoin transactions are created using unspent transaction output (UTXO model)
 - A UTXO has well-defined inputs (one or more) and outputs (one or more)



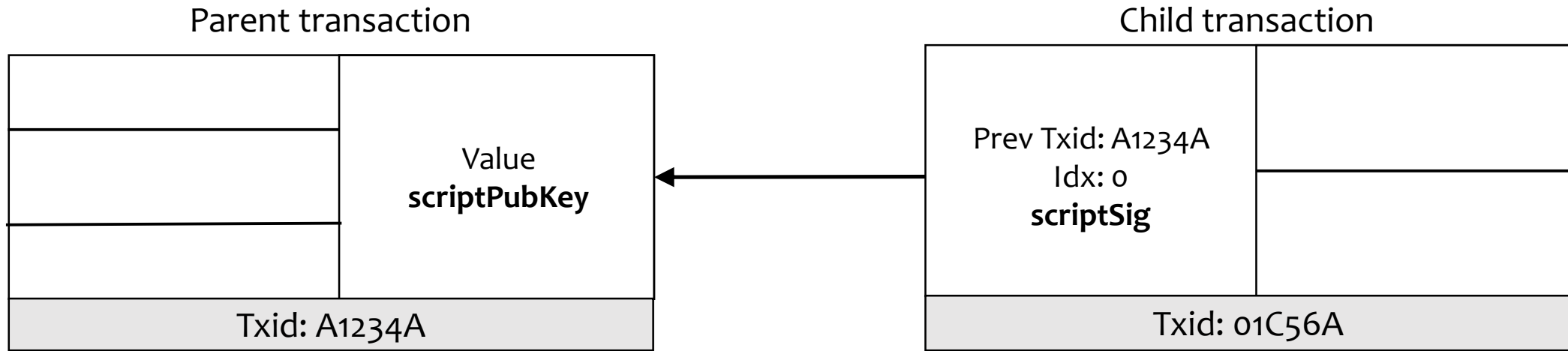
Rules for creating inputs and outputs

- Each output should mention a value and a 'scriptPubKey'
 - **scriptPubKey is the locking script that defines the spending condition for the output**
- Each input must reference exactly one output from a past txn (parent txn)
 - The input contains a 'scriptSig'
 - **scriptSig is the unlocking script for the referenced output in legacy transactions**
 - **scriptSig is empty in SegWit transactions (instead witness field contains the unlocking script)**
- The sum of input values should be greater than or equal to the sum of output values

With these rules, Bitcoin does not require user accounts

Spending an Output

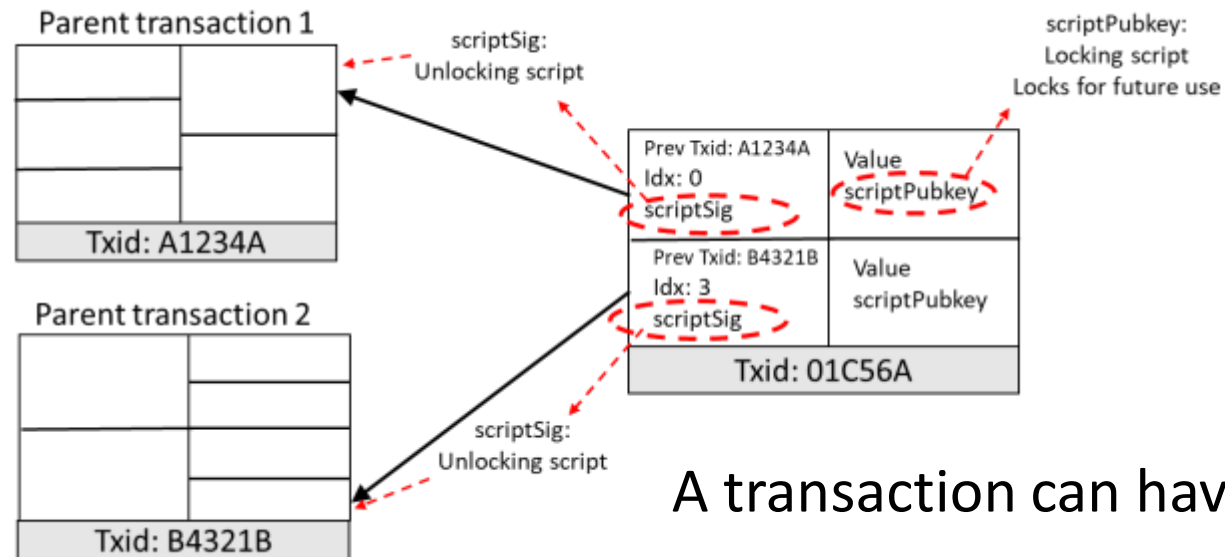
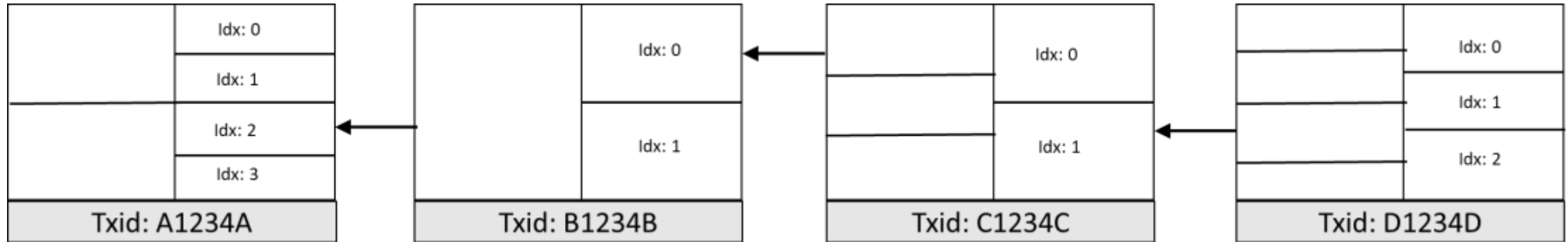
- Parent transaction is also referred to as the funding transaction
- Child transaction is also referred to as the spending transaction



- The value is locked at the **scriptPubKey** field at the output of the parent transaction
- The input of the child transaction identifies the output of the parent transaction that it wants to spend and provides the required **scriptSig** to unlock the output
- scriptSig + scriptPubKey is executed on the stack and if the result is TRUE, the output is unlocked and completely spent via the new outputs of the child transaction

Transaction Chaining

Bitcoin transactions also form a chain: each transaction spends output(s) of previous transaction(s) and creates new outputs for future use



A transaction can have multiple inputs

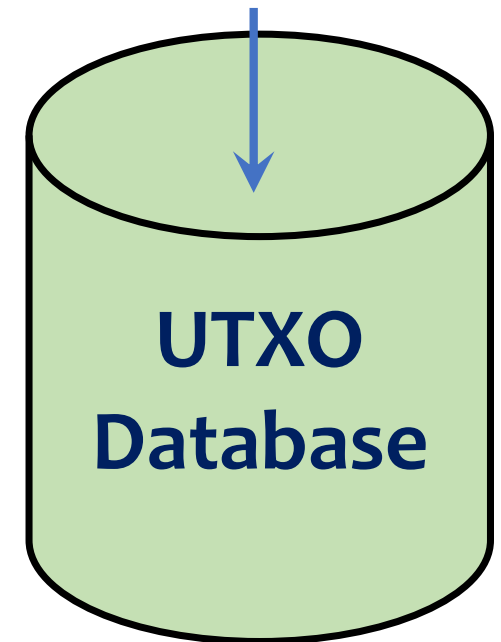
UTXO Database

- UTXOs are scattered in the blocks
 - Nodes maintain a UTXO database (called chainstate.db in Bitcoin core)
 - The purpose of this database is to speed up the transaction validation process

Simplified Transaction Validation & Database Update Algorithm

1. For each input in a new TXN (child txn):
 - a. If parent UTXO is not in the DB, return an error
 - b. If the provided signature on child txn does not match the owner (Pubkeyhash) of the parent UTXO, return an error
2. If the total value of all the referenced parent UTXOs is less than the total value of all the outputs of child txn, return an error
3. If successful, remove all parent UTXOs and add all child UTXOs to the database

Transaction outputs
that have not been
spent at a given
time

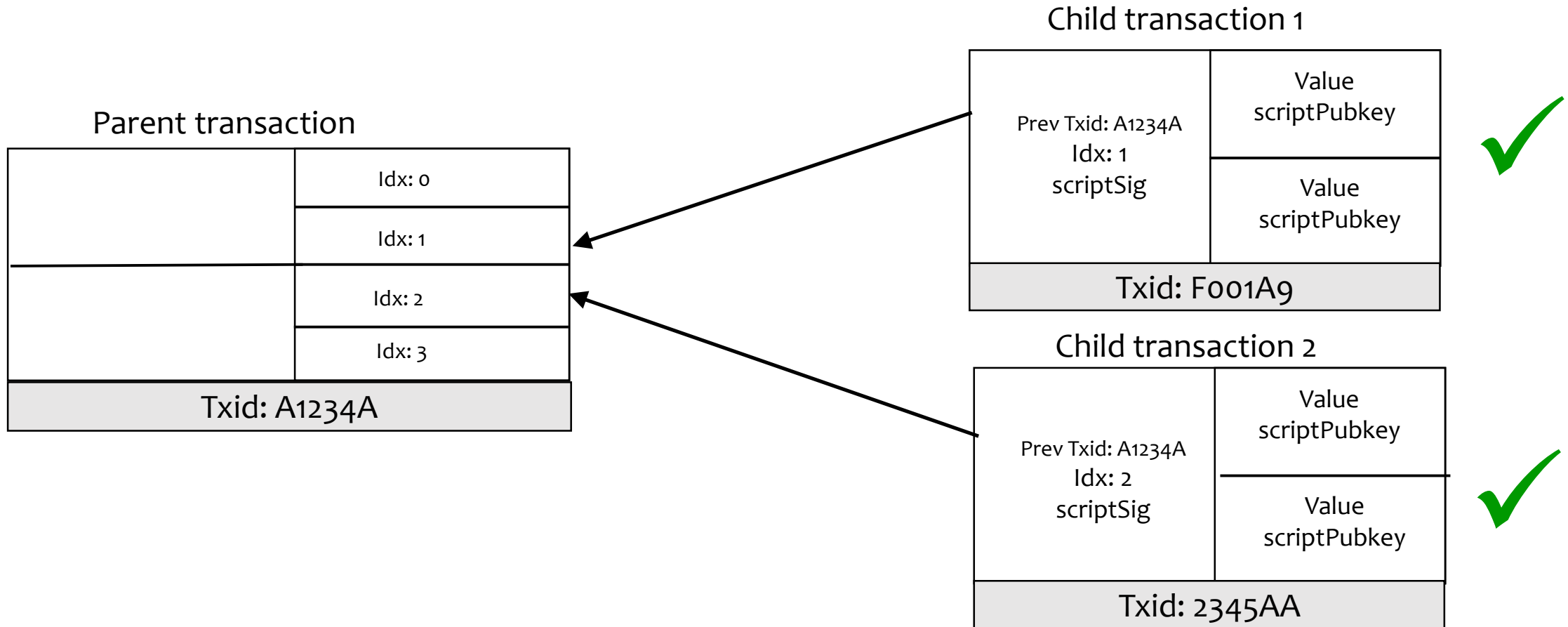


Lecture 14

Thursday, Mar 5, 2026

UTXO Model

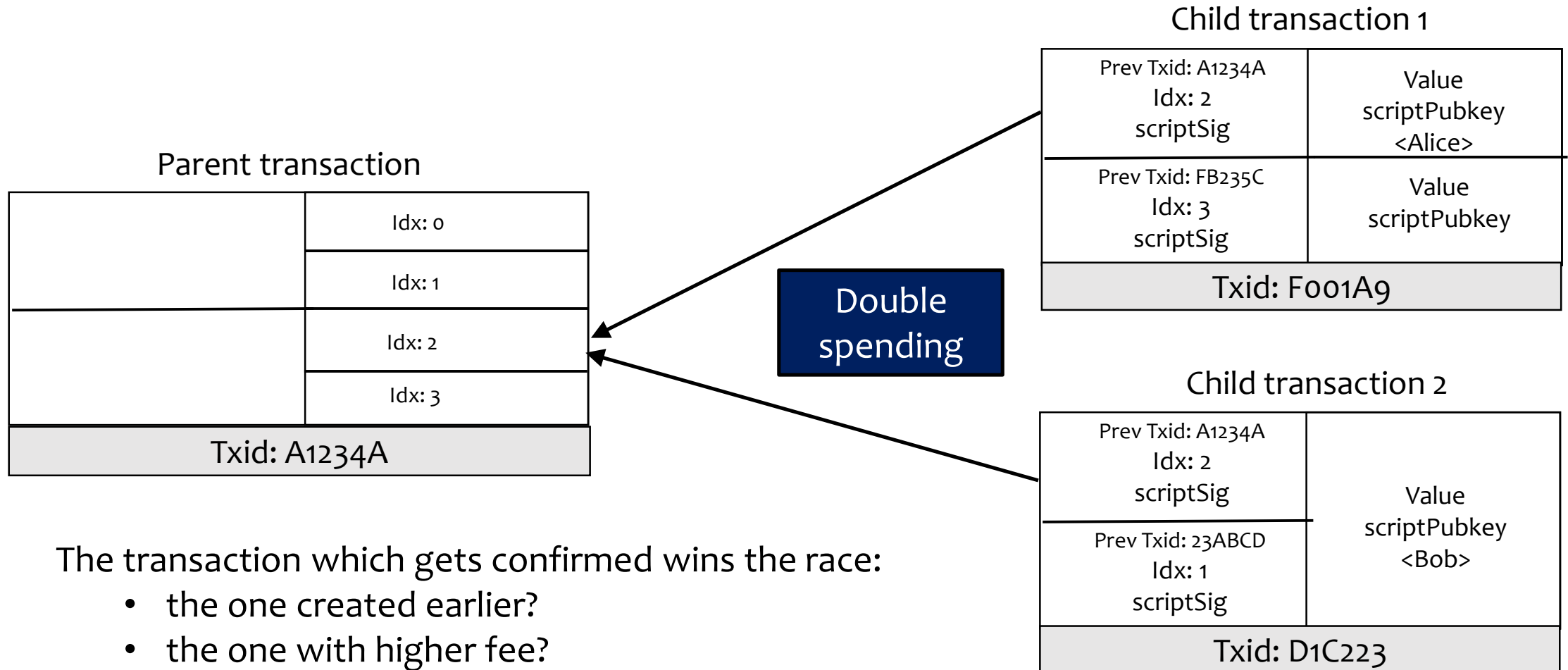
Which child txn is valid assuming signatures match and total value of outputs is less than the value of referenced UTXO?



Both are valid because they are pointing to two different outputs Idx 1 and Idx 2 respectively

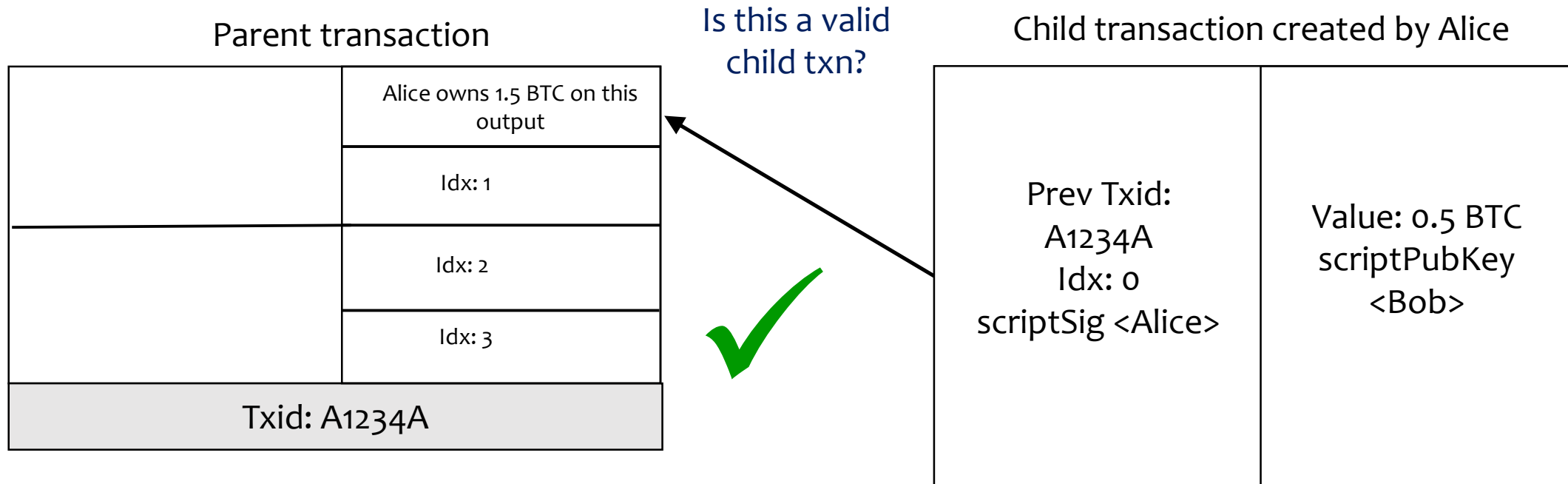
Race between Transactions

The two child transactions are in a so-called ‘race’



Making payments

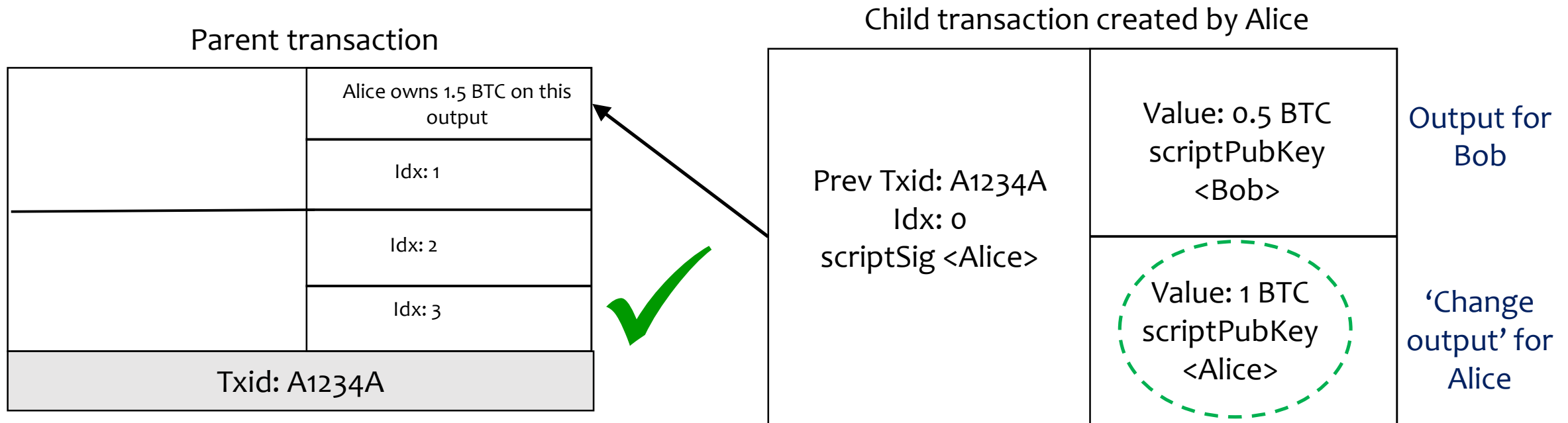
Alice wants to pay 0.5 BTC to Bob. Alice owns 1.5 BTC at index 0 of Txid: A1234A



Alice owns a valid UTXO, and sum of inputs > sum of outputs, so this is perfectly fine

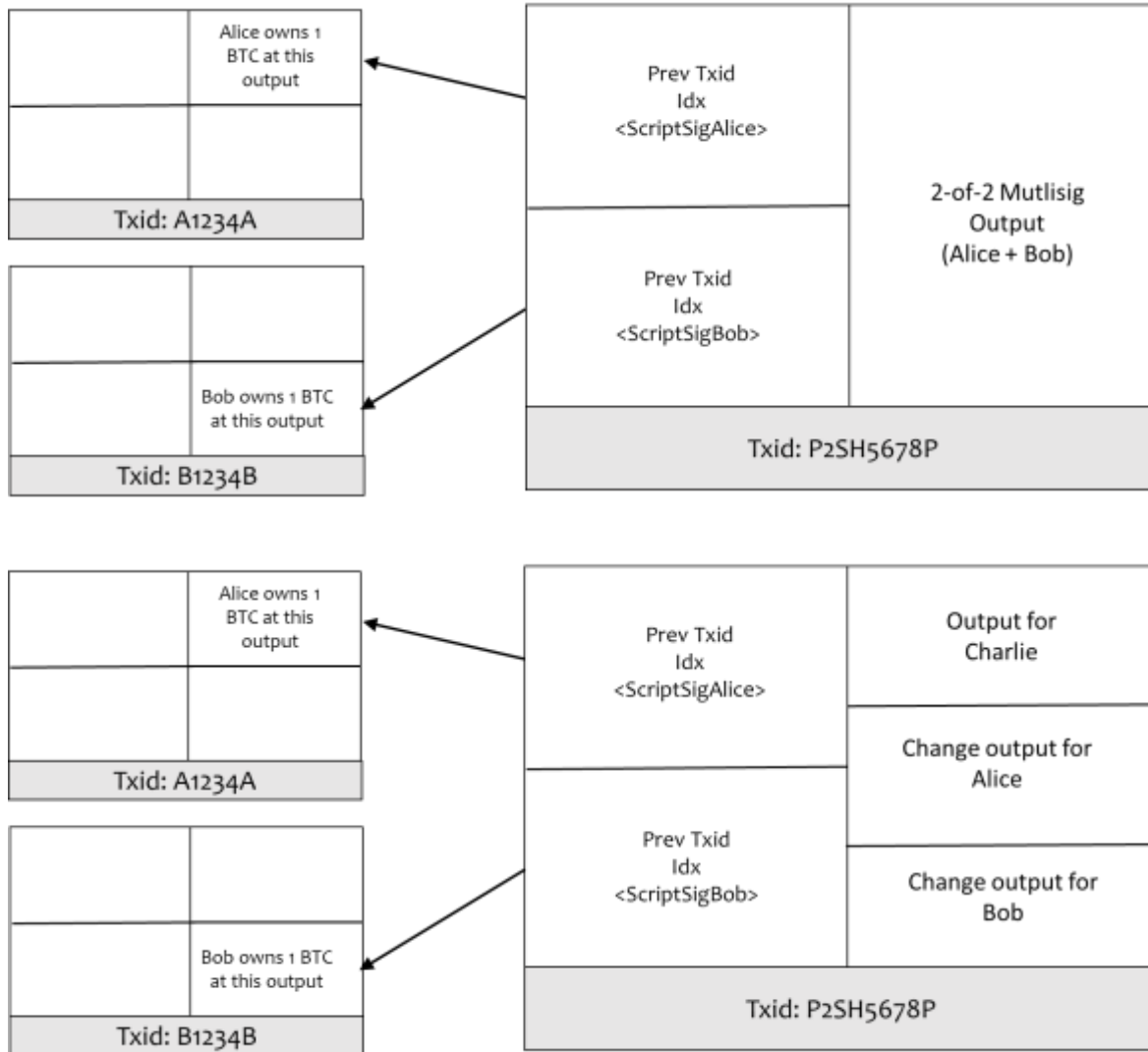
Any difference between the inputs and outputs is retained by the miner as txn fee so Alice will lose 1 BTC in this example

Payment with a 'Change Output'



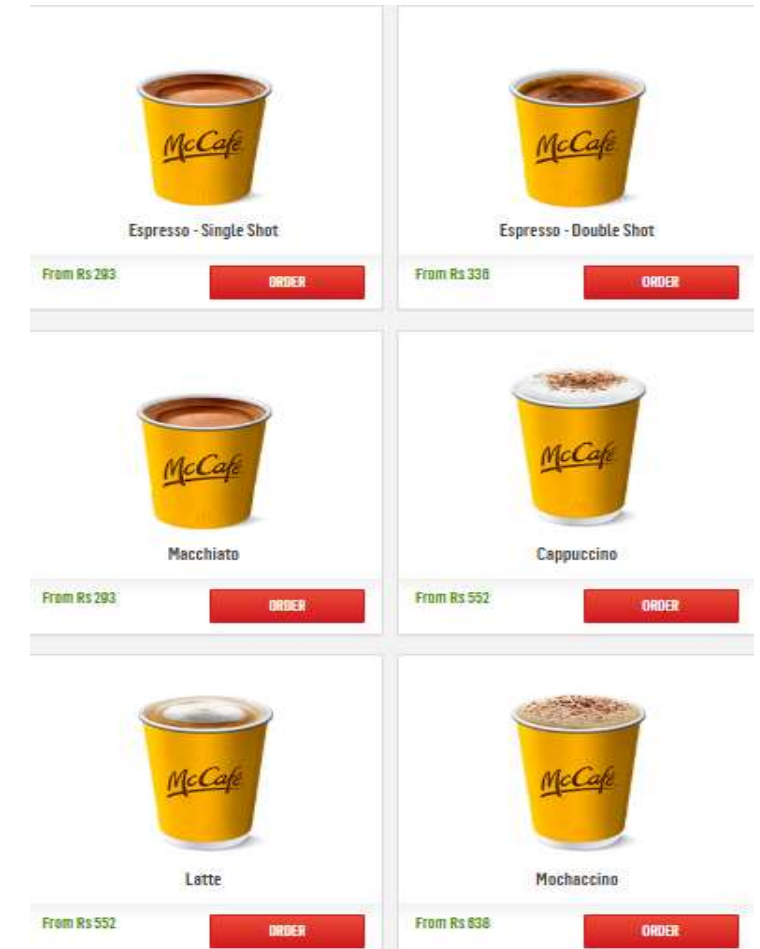
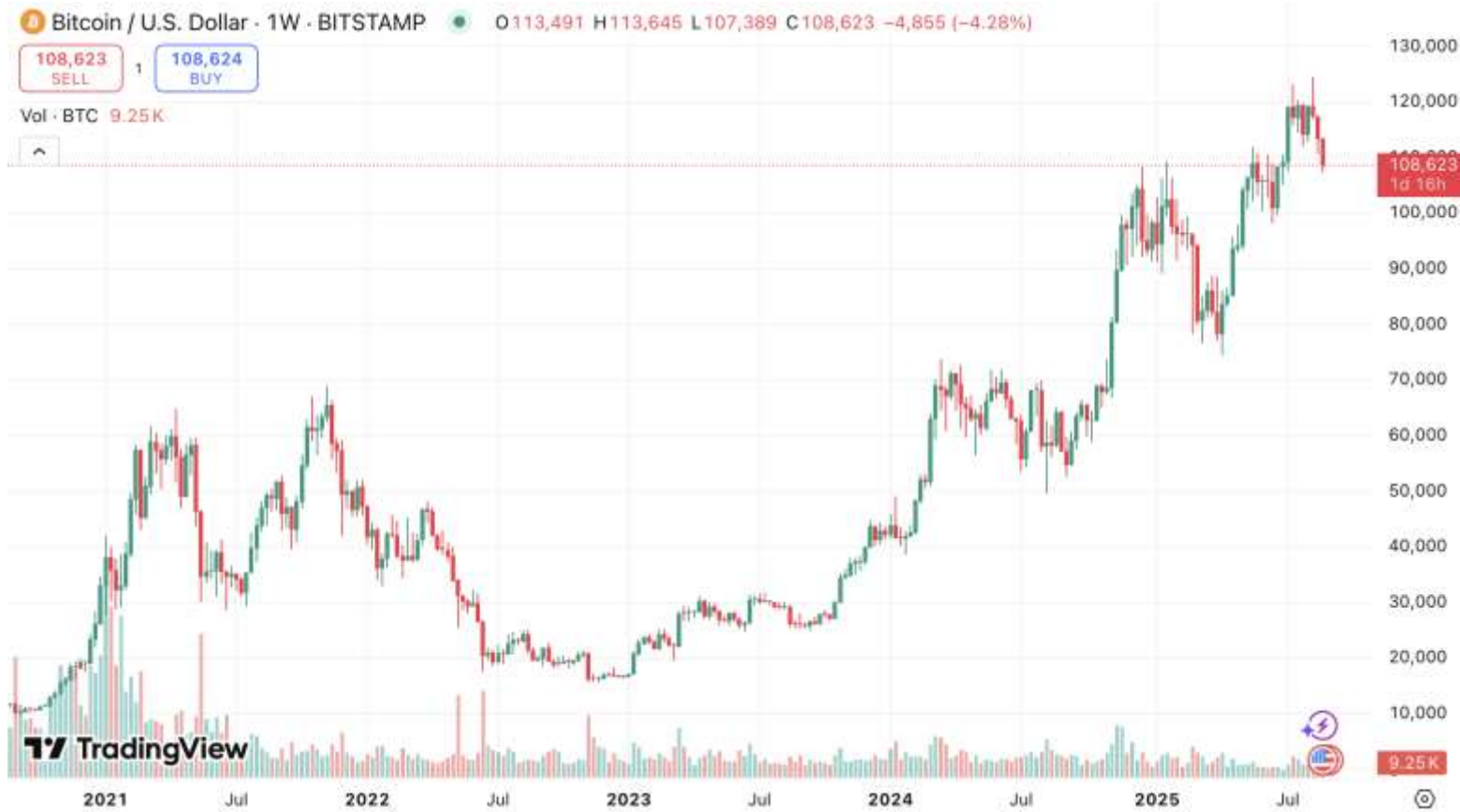
- Alice must pay herself the remaining BTC by creating a second output – this is equivalent to receiving a change for her future use
- This is a typical Bitcoin payment transaction with two outputs – one payment output to the intended recipient, and a change output returning the remaining funds to the sender

Collaborative Transaction



- Alice and Bob collaborate and agree on all the inputs and all the outputs (value, spending conditions)
- Either Alice or Bob creates an ‘*unsigned transaction*’
 - Unsigned transaction is shared with other party
- Alice signs the unsigned transaction (all the inputs and all the outputs) with her private key (produces <SigAlice> for this txn)
- Bob signs the unsigned transaction (all the inputs and all the outputs) with his private key (produces <SigBob> for this txn)
- To obtain a fully signed transaction, both parties exchange their signatures
 - The party holding all the signatures has the fully-signed transaction
 - The party with only some signatures holds a so-called partially signed bitcoin transaction (PSBT)
- Only the fully signed transaction can be broadcast to the Bitcoin network

BTC Price vs a Cup of Coffee



Bitcoin seems better suited for buying the entire coffee shop than a single cup!

Start Thinking in Satoshis!

- For micropayments, let us start thinking in Satoshis (sats)
- 1 BTC = 10^8 sats (100 million)
- Assume BTC price is 100,000 \$ (10^5 \$)
- 1 sat = $10^5 / 10^8 = 1/1000 = 0.001$ \$ (0.3 pkr)
- 1 cup of coffee is approximately 1 \$ or 1000 sats

Satoshi to USD / BTC Converter & Calculator

Currency	Amount
 Satoshis SATS	871
Currency	Amount
 US Dollar USD	1.00
Currency	Amount
 Bitcoin BTC	0.00000871

<https://bitbo.io/tools/satoshi-usd/>

Now bitcoin starts to feel like a usable currency

Transaction Fee

- 1 cup of coffee is approximately 1 \$ or 800 to 1000 sats
- Average transaction fee is around 0.5 to 1.5 \$ ~ 400 to 800 sats



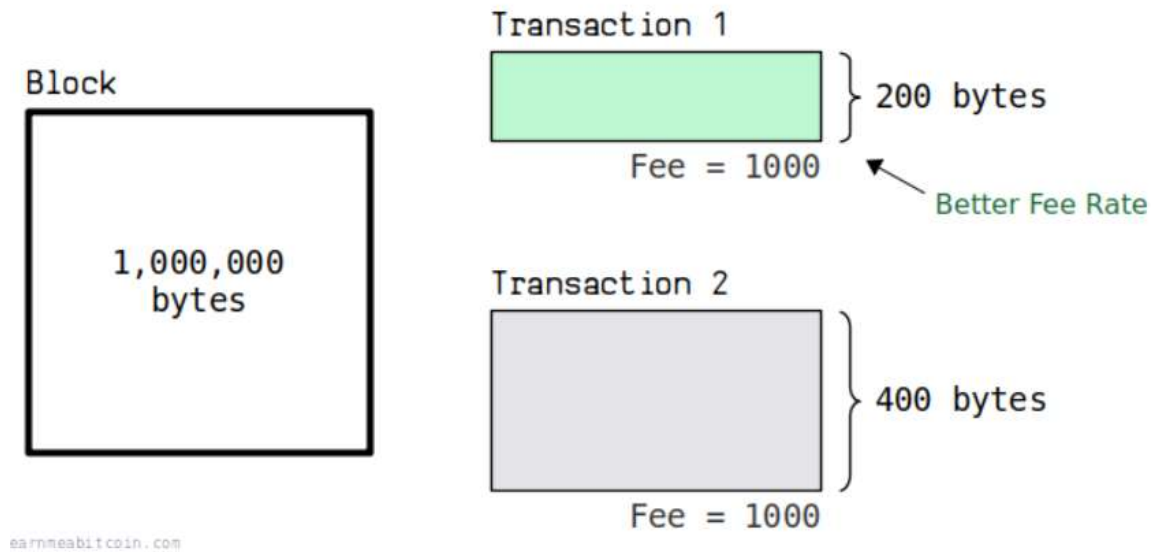
Bitcoin Transaction Cost Estimate

Transaction Type	Median sat/vB	Size [vB]	Cost [sats]
P2TR (Taproot) 2 Inputs + 2 Outputs	2.056	212	435
P2TR (Taproot) 2 Inputs + 3 Outputs	2.056	255	523
P2WPKH 1 Input + 2 Outputs	2.056	141	289
P2WPKH 2 Inputs + 2 Outputs	2.056	209	429
P2WPKH 2 Inputs + 3 Outputs	2.056	240	492
P2PKH 1 Input + 2 Outputs	2.056	226	465
P2PKH 2 Inputs + 2 Outputs	2.056	374	769
P2PKH 2 Inputs + 3 Outputs	2.056	408	839

It still seems difficult to use BTC for micropayments due to high transaction fee!

Transaction Fee Depends on Size

- In Bitcoin, transaction fee does not depend on the transaction value
- Transaction fee depends on the transaction size
 - Large sized transactions (more bytes) are costly
- A 1 \$ coffee payment transaction can end up paying the same fee as a 100,000 \$ BTC transfer if the transaction size in bytes is the same



Can we exploit this property to reduce the fees for micropayments?

Reducing the Fee for Micropayments

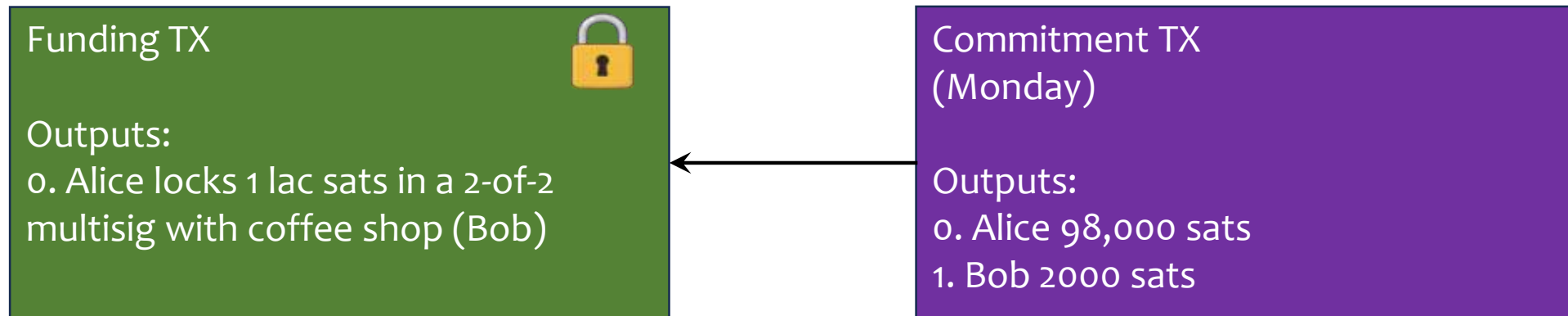
- Suppose we make a single high-value transaction with an output of 1 lac sats
 - This transaction is confirmed on blockchain
 - We pay the transaction fee (e.g., 500 sats)
 - We call it the ‘Funding transaction’
- We use the funding transaction to make several micropayments
 - After every micropayment, we only update the balances off-chain by creating ‘commitment transactions’
 - After several micropayments, we settle the final balances on-chain
 - We pay the transaction fee for the last commitment transaction that is confirmed on the blockchain (e.g., 500 sats fee)
- We pay the fee for the initial funding transaction and the last commitment transaction as they enter the bitcoin blockchain (only 1000 sats in transaction fees)

Commitment transactions exist off-chain, so the challenge is to make them as secure as the on-chain Bitcoin transaction

Funding & Commitment Transactions

Funding transaction:
2-of-2 multisig transaction on blockchain
used to lock funds in shared ownership

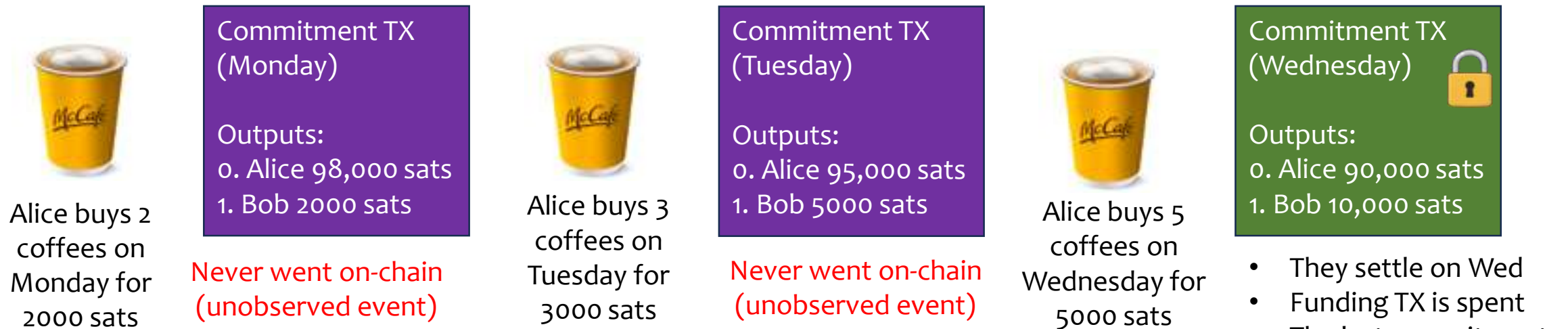
Commitment transaction:
A transaction or a number of transactions
that return the funds of a two-party
channel to their owner



- Before publishing the funding transaction, the funder first obtains a pre-signed spending transaction from the counterparty so the funds can always be recovered even if cooperation stops
- Once the funding transaction is safe to publish, the peers move forward by creating the commitment transactions

Unobserved Events

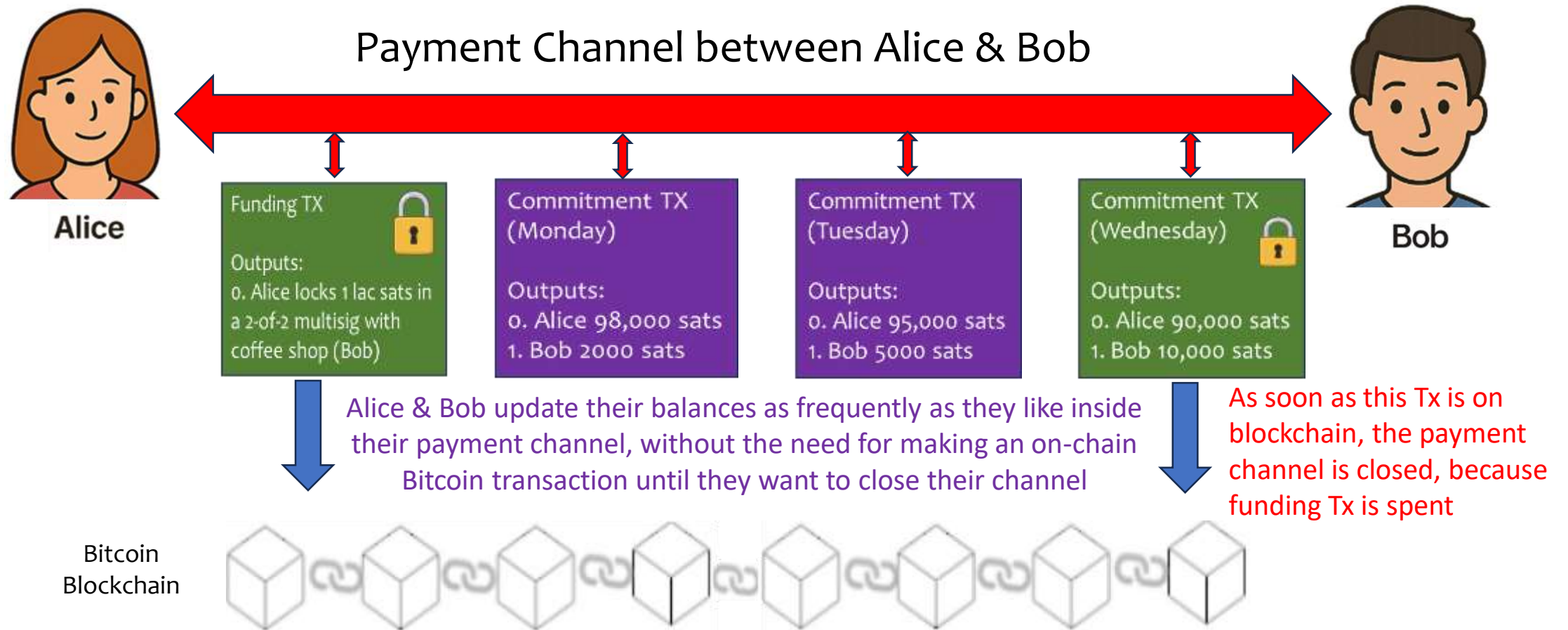
- After every coffee purchase, Alice & Bob create new commitment transactions and update their balances
- Either Alice or Bob can enforce the settlement by broadcasting and enforcing the latest agreed state without needing cooperation from the other party



All the intermediate transactions remain unobserved, which is also good for privacy

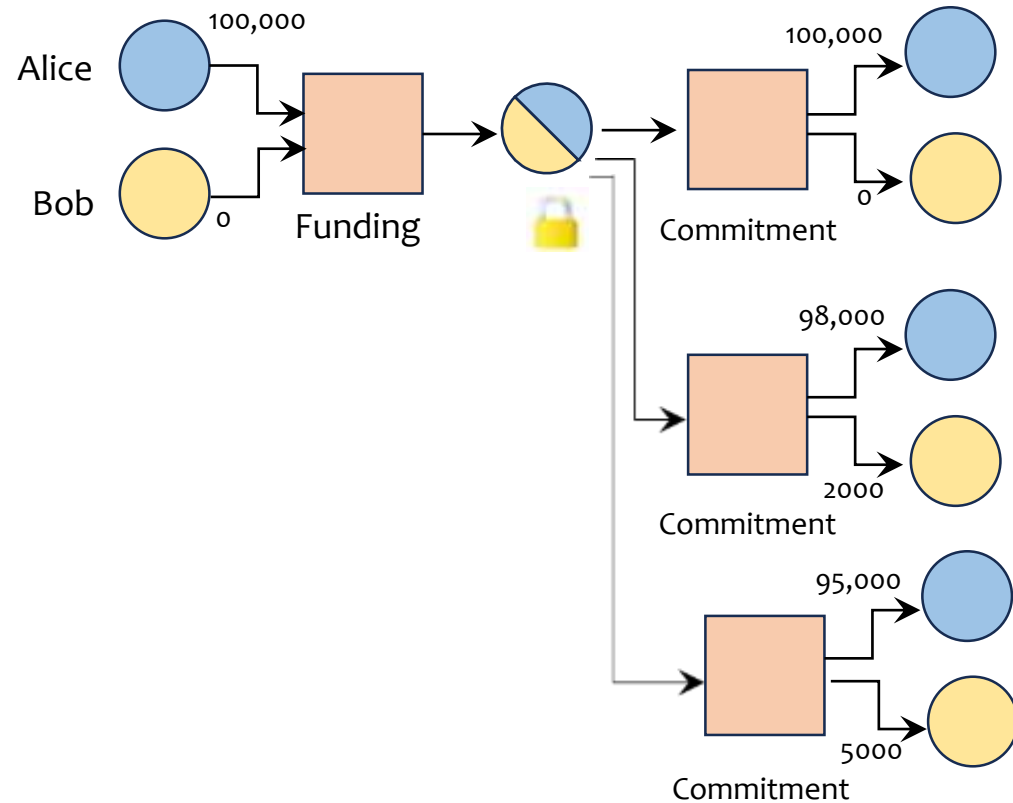
Payment Channel

Payment Channel is a setup that allows two parties to make payments without contacting the blockchain



Making this payment channel secure and trustless is a major challenge

Payment Channel



1. Alice & Bob decide to open a payment channel
2. Suppose Alice funds the channel with 100,000 sats
3. Alice & Bob create a 2-of-2 multisig funding tx
4. Bob signs a commitment transaction to pay back Alice
5. Once Alice has the commitment transaction, the funding transaction is broadcast and confirmed on-chain
6. Alice buys coffee and together with Bob decides to update the channel
7. Alice & Bob create a new commitment transaction with updated balances
8. Alice buys another coffee and together they decide to update the channel
9. They update their balances in the new commitment transaction
10. The process continues till they decide to close the channel

Lecture 15

Tuesday, Mar 24, 2026

Security Risks

Each commitment transaction is a valid bitcoin transaction, but it is not posted on-chain in usual circumstances

What if one party tries to cheat and broadcasts an old commitment transaction?

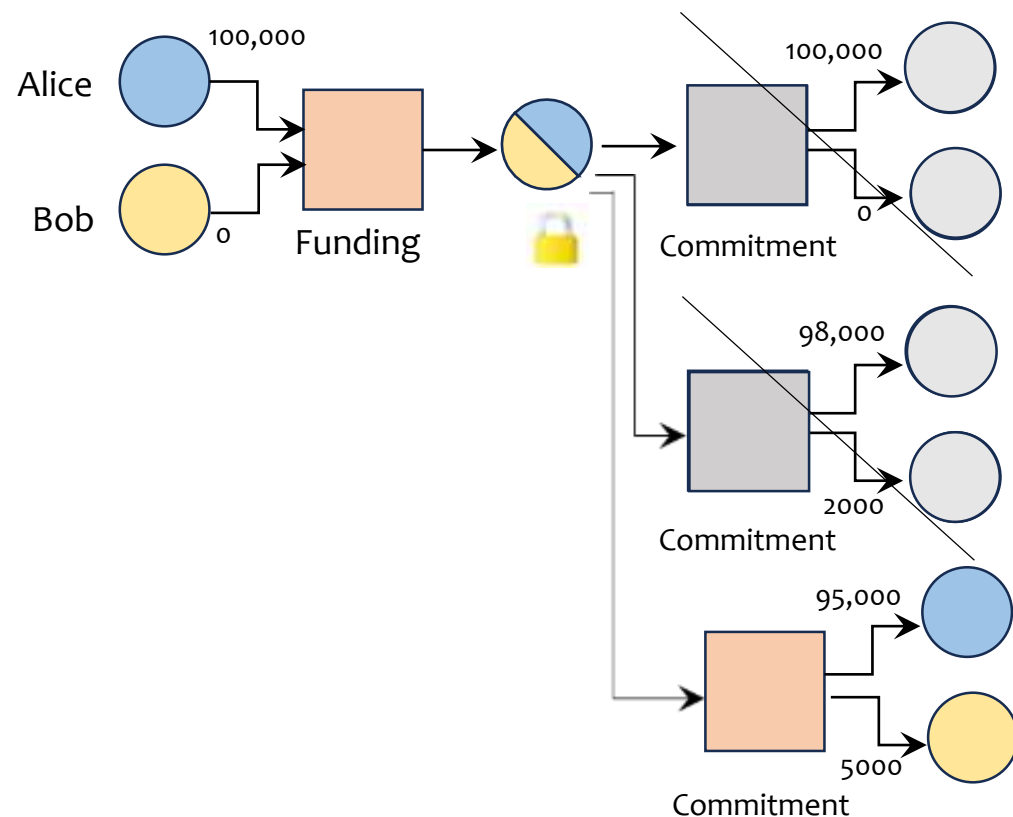
What if one party disappears or goes offline?

What measures are required to ensure a trustless payment channel?

Preventing Old Commitment Publishing

- All the old commitment transactions are valid transactions
 - Peers can try to publish old commitment transactions that award them a higher balance
- It is therefore necessary to prevent old commitment transactions from being broadcast for this method to work in practice

Trustless Payment Channel



1. Alice & Bob decide to open a payment channel
2. Suppose Alice funds the channel with 100,000 sats
3. Alice & Bob create a 2-of-2 multisig funding tx
4. Bob signs a commitment transaction to pay back Alice
5. Once Alice has the commitment transaction, the funding transaction is broadcast and confirmed on-chain
6. Alice buys coffee and together with Bob decides to update the channel
7. Alice and Bob revoke the old commitment
8. Alice & Bob create a new commitment transaction with updated balances
9. Alice buys another coffee and together they decide to update the channel
10. They revoke the old commitment & update their balances in the new commitment
11. The process continues till they decide to close the channel

Fundamental Challenges

- There is no way to cancel a valid Bitcoin transaction
- However, to make payment channels work, we need to find a way to revoke old commitments
- There are two fundamental challenges that need to be addressed to create trustless payment channels

How to ascribe blame for broadcasting an old transaction?

How to penalize the party that tries to publish the old commitment?

Assigning Blame

- Create two different commitment transactions for every update
 - Commitment transaction *C1a* for Alice which only she can broadcast
 - Commitment transaction *C1b* is for Bob which only he can broadcast
- Both *C1a* and *C1b*
 - Spend the same funding transaction (same parent)
 - Reflect the same balance split between Alice & Bob
 - Outputs are constructed differently, which makes them uniquely identifiable
- If *C1a* appears on-chain, everyone knows Alice published it
- If *C1b* appears on-chain, everyone knows Bob published it

Creating two different commitment transactions solves the blame assignment problem

Add Asymmetric Time Delays

- To make the commitment transactions different and assign blame correctly, we add asymmetric time delays in the outputs according to the following rule:
 - Whoever broadcasts a commitment transaction (old or new) must wait before spending their funds, while the counterparty can immediately spend

- C1a (held by Alice) spends from the funding transaction and has two outputs:
 - Alice's output is time-locked (she must wait to redeem it)
 - Bob's output is immediately spendable

Commitment TX (Alice can broadcast)
(C1a)

Outputs:

0. Alice 98,000 sats (time locked)
1. Bob 2000 sats

- C1b (held by Bob) spends from the funding transaction and has two outputs:
 - Alice's output is immediately spendable
 - Bob's output is time-locked (he must wait to redeem it)

Commitment TX (Bob can broadcast)
(C1b)

Outputs:

0. Alice 98,000 sats
1. Bob 2000 sats (time locked)

Commitment Transaction Creation

- Bob creates C1a for Alice
 - Bob creates an output for himself that is immediately spendable
 - Bob creates an output for Alice with an agreed upon time lock
 - Bob signs the transaction & gives the half-signed transaction to Alice
 - Alice adds her own signature but never gives the fully signed C1a back to Bob
 - Since the funding transaction is 2-of-2 multisig so Bob cannot broadcast the half-signed C1a
 - Therefore, if C1a ever appears on the network, everyone knows that it must have come from Alice
- Alice creates C1b for Bob using similar procedure

Neither party can fabricate a different version of C1a or C1b because each commitment requires both signatures

Do We Need SegWit?

Prior to SegWit, transaction id (Txid) was computed by taking the hash of all the transaction data (signature included) in a well-defined serialization order

Legacy Txid=SHA256(SHA256(All txn data))

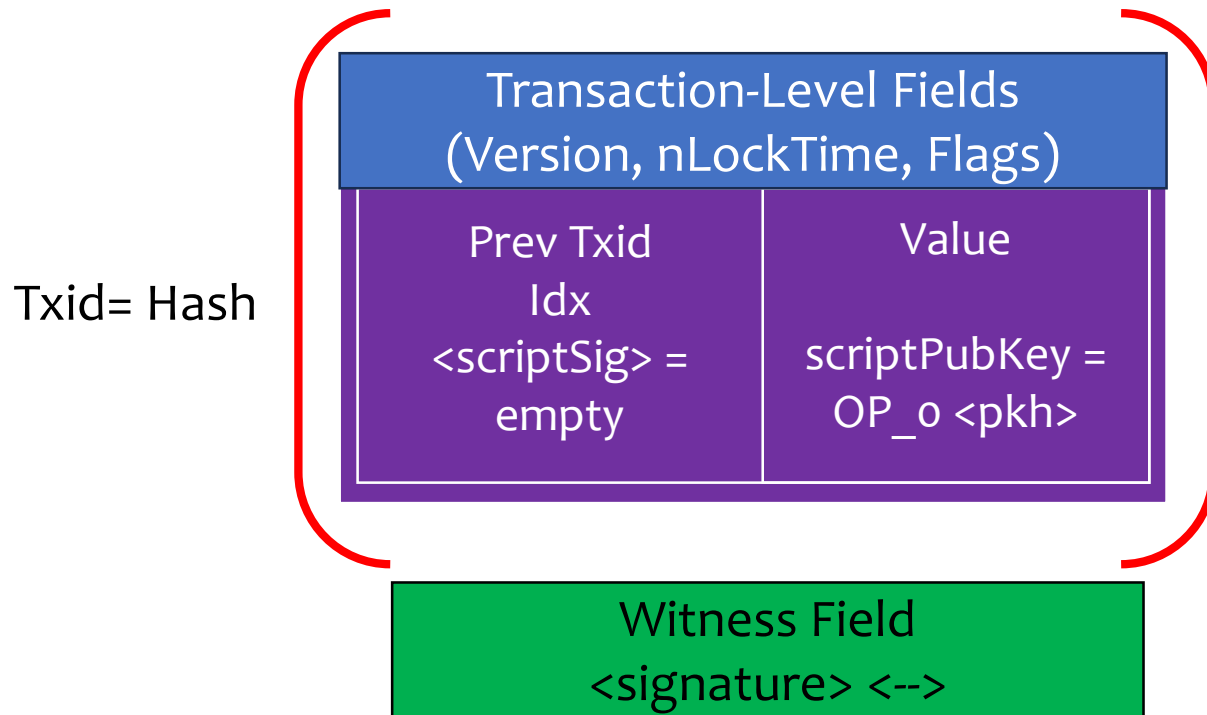
Legacy Txid = Hash

Transaction-Level Fields (Version, nLockTime, Flags)	
Prev Txid: Funding Tx Idx: 0 <scriptSig> = <sigAlice> <sigBob>	Alice 98,000 sats (time locked) Bob 2000 sats (no time lock)

- Txid identifies a transaction for referencing it on blockchain network
- A Merkel tree of all the Txids in a block is also created and its root is stored in the block header

- Legacy transaction format cannot be used for payment channels because when signatures are part of the inputs, both signatures are required to compute the txid
- If someone malleates the txid, the child transactions that spend those commitment transactions become invalid

SegWit Solves the Problem



$\text{Txid} = \text{SHA256}(\text{SHA256}(\dots))$

Txid cannot be malleated because
scriptSig is kept empty

Witness field contains the signatures as well as any
other spending condition(s)

- Not considered part of the txn so not used in computing Txid
- Stored in the block but it is not committed directly into the Merkel root of Txids in the block header
- scriptPubKey is treated as a witness program

- In our payment channel, to assign blame, Bob creates C1a for Alice and provides the signatures separately
- Removing signatures from the input allows Bob to know the Txid before Alice even signs it
- Bob can safely link his child transactions to Alice's commitment transactions

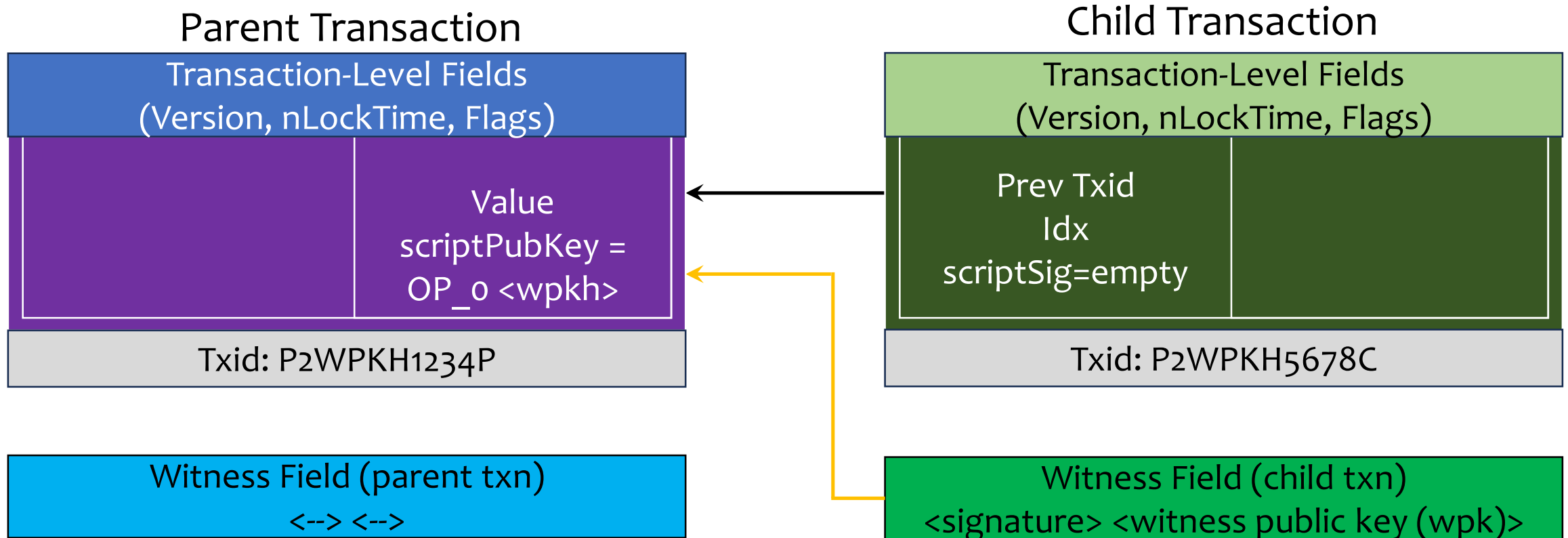
Lecture 16

Thursday, Mar 26, 2026

P2WPKH

P2WPKH (Pay-to-Witness-Public-Key-Hash)

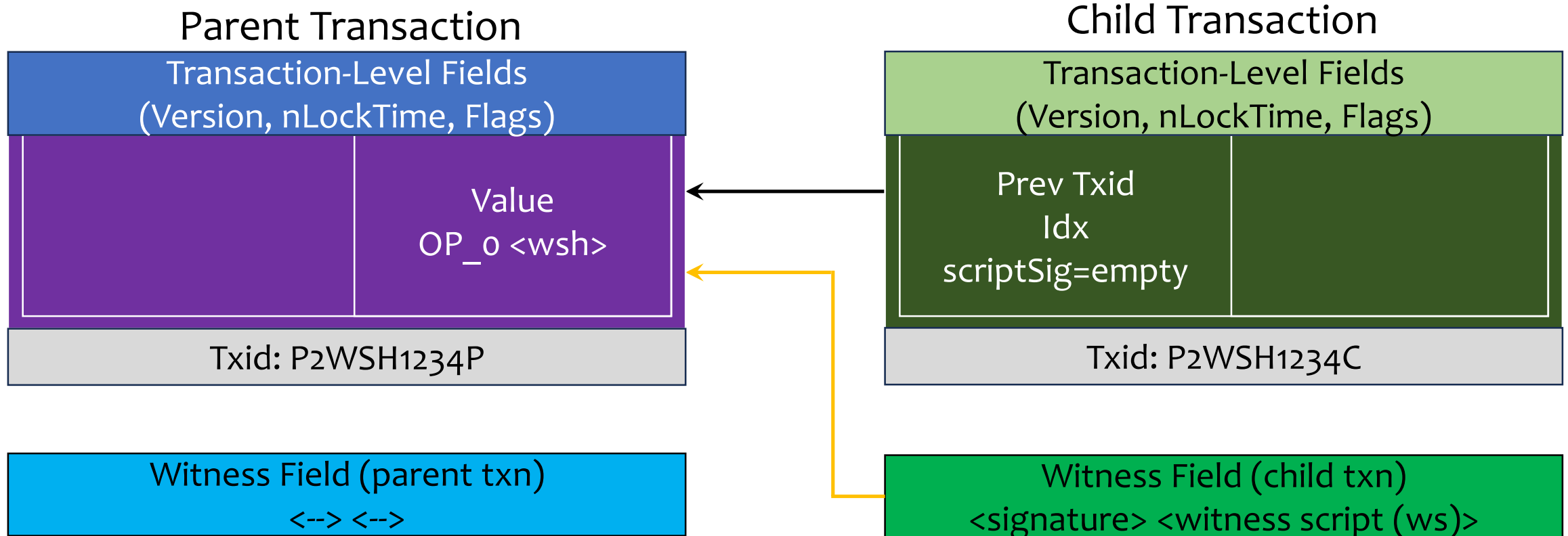
- Output (parent txn): `<scriptPubKey> = OP_0 <wpkh>`
- Input (child txn): `<scriptSig> = empty`
- Witness (child txn): `<sig> <wpk>`



P2WSH

P2WSH (Pay-to-Witness-Script-Hash)

- Output (parent txn): OP_o <wsh>
- Input (child txn): <scriptsig> = empty
- Witness (child txn): <sig> <ws>



SegWit Transaction Validation

Transaction validation by the Old Nodes

- Old nodes run the older version of the software that recognizes the legacy transaction format
- The old nodes find the signature field empty
 - They validate without the signature because they do not find it anywhere inside the transaction
 - Old nodes validate SegWit transactions as “anyone-can-spend”
- In the output script, the old nodes first pop a zero (OP_0) on the stack followed by <wpkh> or <wsh>
 - A non-zero number at the top of the stack is interpreted as *True*, therefore, validation succeeds

Transaction Validation by New Nodes

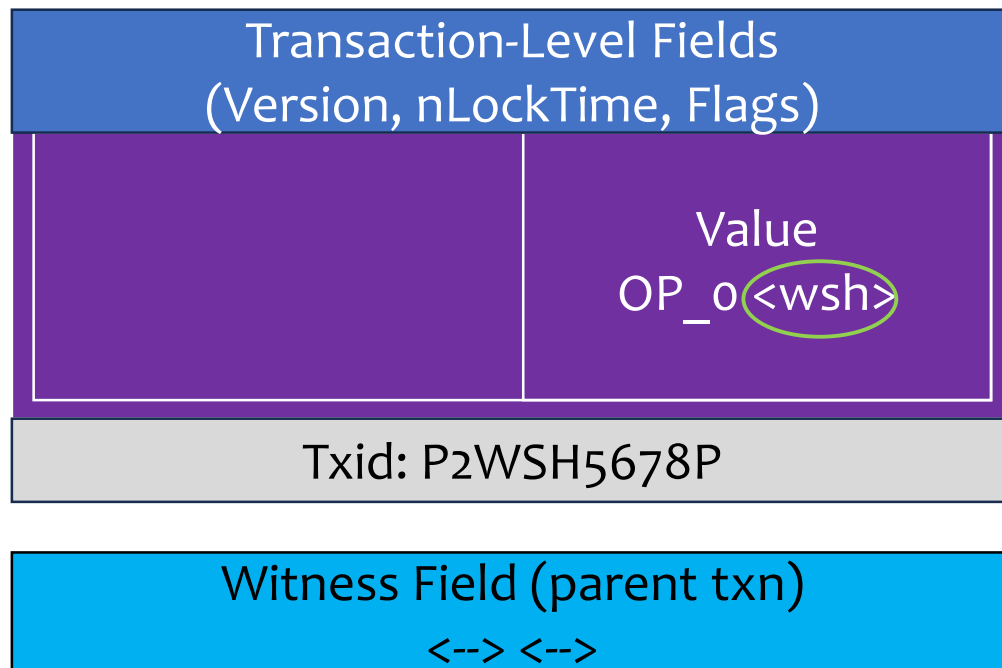
- New nodes see OP_0 in the scriptPubKey, which indicates that it is a SegWit output
- They check the witness field and execute the witness program
- The type of program depends on the length of data in the scriptPubKey
 - 20B – they run p2wpkh program that verifies the hash of public key in the witness field
 - 32B – they run p2wsh program that verifies the hash of witness script and also runs the script

Conditional Spending Rules using P2WSH

Conditional spending rules are implemented using P2WSH

Suppose we want to create a **P2WSH** output that Alice can spend only with her signature (private key), while Bob can spend with a secret S whose hash is $H = \text{SHA256}(S)$

Parent Transaction



Witness Script (its hash goes into the scriptPubKey field of the parent txn):

OP_IF

<PubKeyAlice> OP_CHECKSIG

OP_ELSE

OP_SHA256 <H> OP_EQUALVERIFY

<PubKeyBob> OP_CHECKSIG

OP_ENDIF

<wsh> = $\text{SHA256}(\text{Witness Script})$ – 32B hash is used in the scriptPubKey of the parent txn

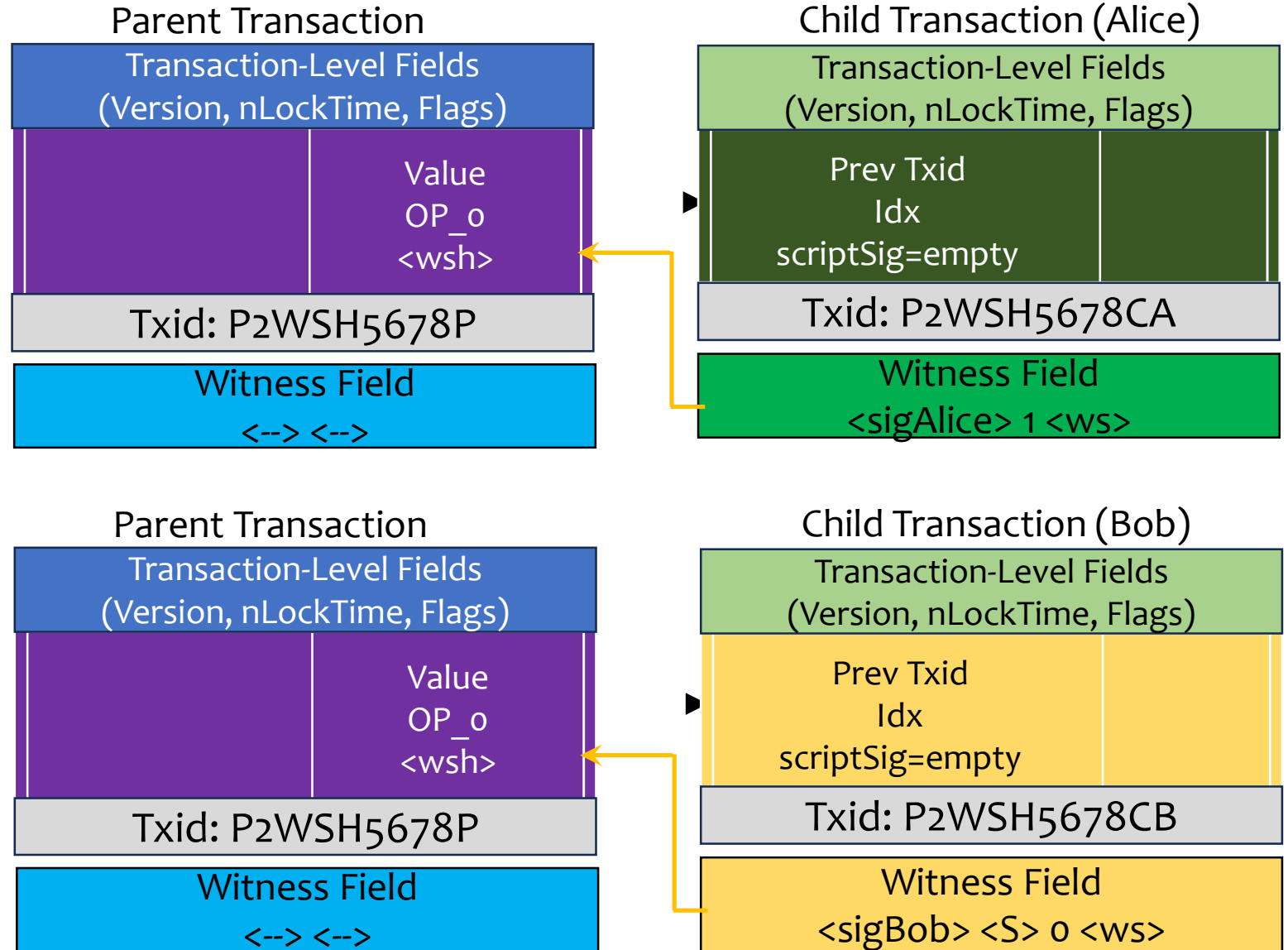
Spending Transactions – P2WSH

Witness Script (ws):

```
OP_IF
  <PubKeyAlice> OP_CHECKSIG
OP_ELSE
  OP_SHA256 <H> OP_EQUALVERIFY
  <PubKeyBob> OP_CHECKSIG
OP_ENDIF
```

<wsh> = SHA256(ws)

- Alice uses 1 in her scriptSig because she can spend via the IF branch according to the witness script
- Bob uses 0 in his scriptSig because he can spend via the ELSE branch according to the witness script
- Bob also provides the secret <S> in his witness field
- **Only one child transaction will be confirmed and as soon as it gets into a block, the other one is invalidated because double spending is not allowed**

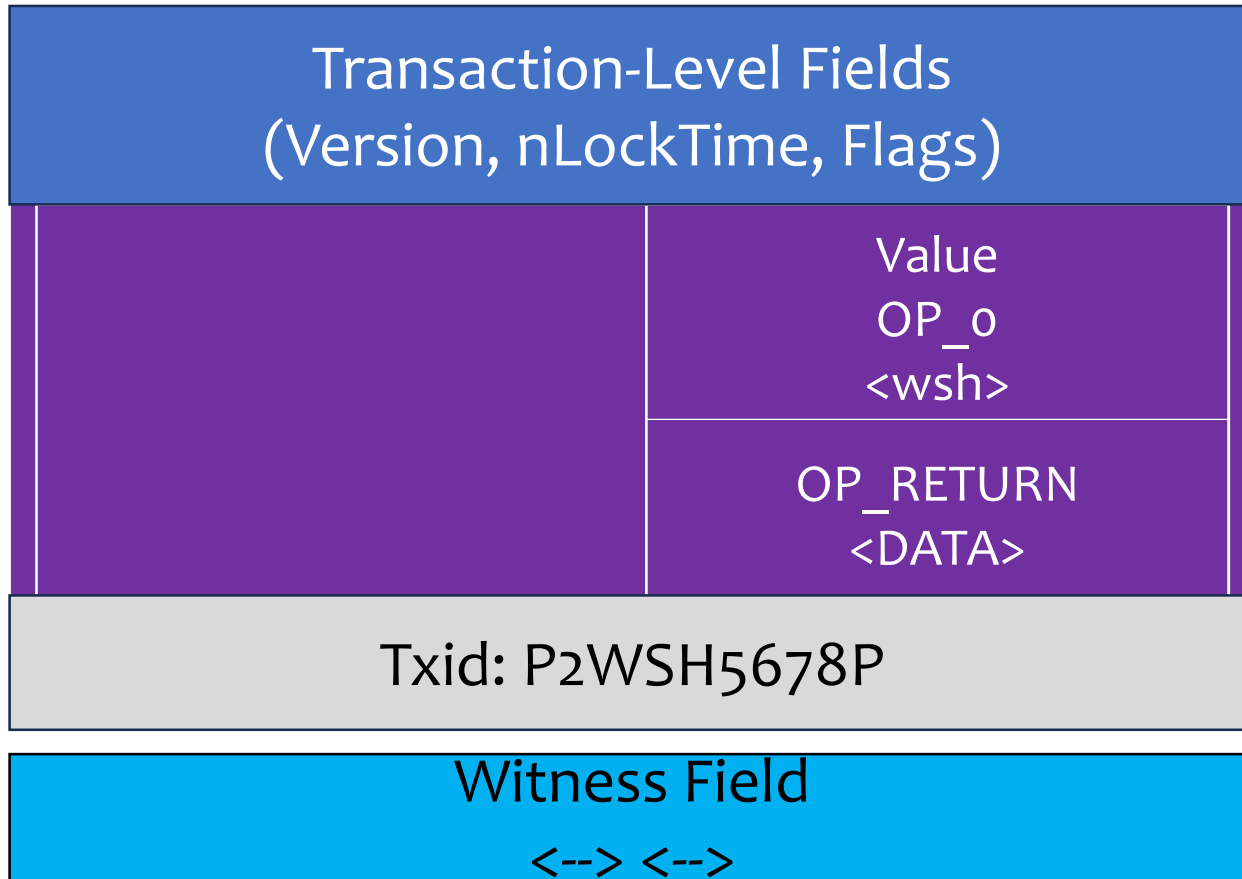


Custom Locking using P2WSH

- Design the Witness Script
 - Write the spending rules in Bitcoin Script (OP_IF / OP_ELSE, OP_SHA256, OP_CHECKSIG, OP_CHECKMULTISIG)
 - Arrange operations & checks so they are in the required order
- Compute the Witness script hash
 - `<sh> = SHA256(Witness Script)`
 - In P2WSH we use SHA256 hash of the witness script
- Create the scriptPubKey
 - `OP_o <wsh>`
 - Use it to lock the value in a parent transaction, `OP_o` indicates it is a segWit program
- Create a child transaction for spending the output
 - Create/gather the required data according to the path used for unlocking
 - Create the Witness (the required sequence of unlocking data items must be followed)
- To validate & execute, nodes interpret it is a witness program when they find `OP_o` in the scriptPubKey of the parent and the 32B length tells them it is a p2wsh program
 - Nodes first check if the witness script in the child txn hashes to the correct value `<wsh>`
 - If correct, the witness script also gets executed (special handling)
 - Finally, the signatures are verified

Storing Data on Bitcoin Blockchain

OP_RETURN is used to store arbitrary data on blockchain

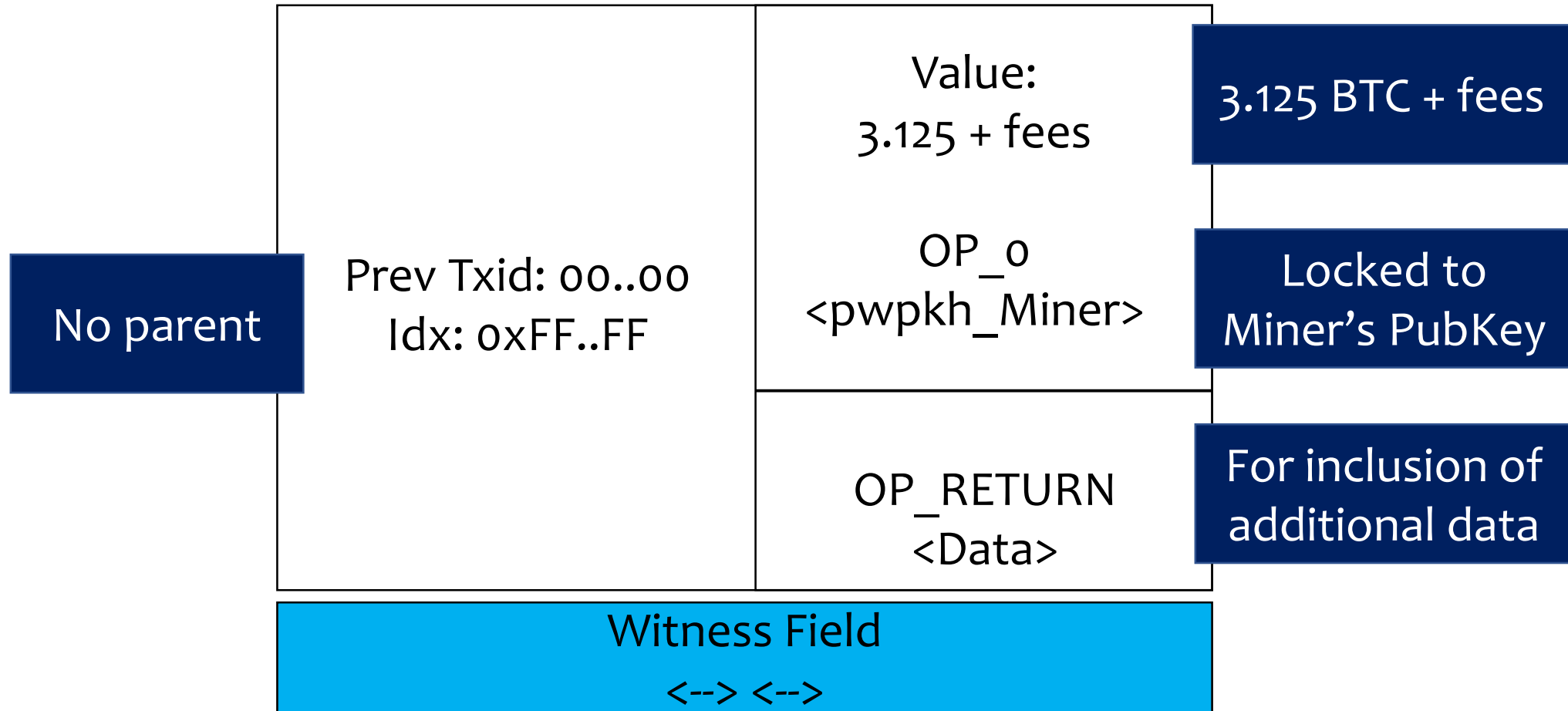


OP_RETURN creates a valid data transaction output that is provably unspendable, so it is not stored in the UTXO database

Any output with OP_RETURN is known to contain data, so these outputs are not stored in the UTXO database

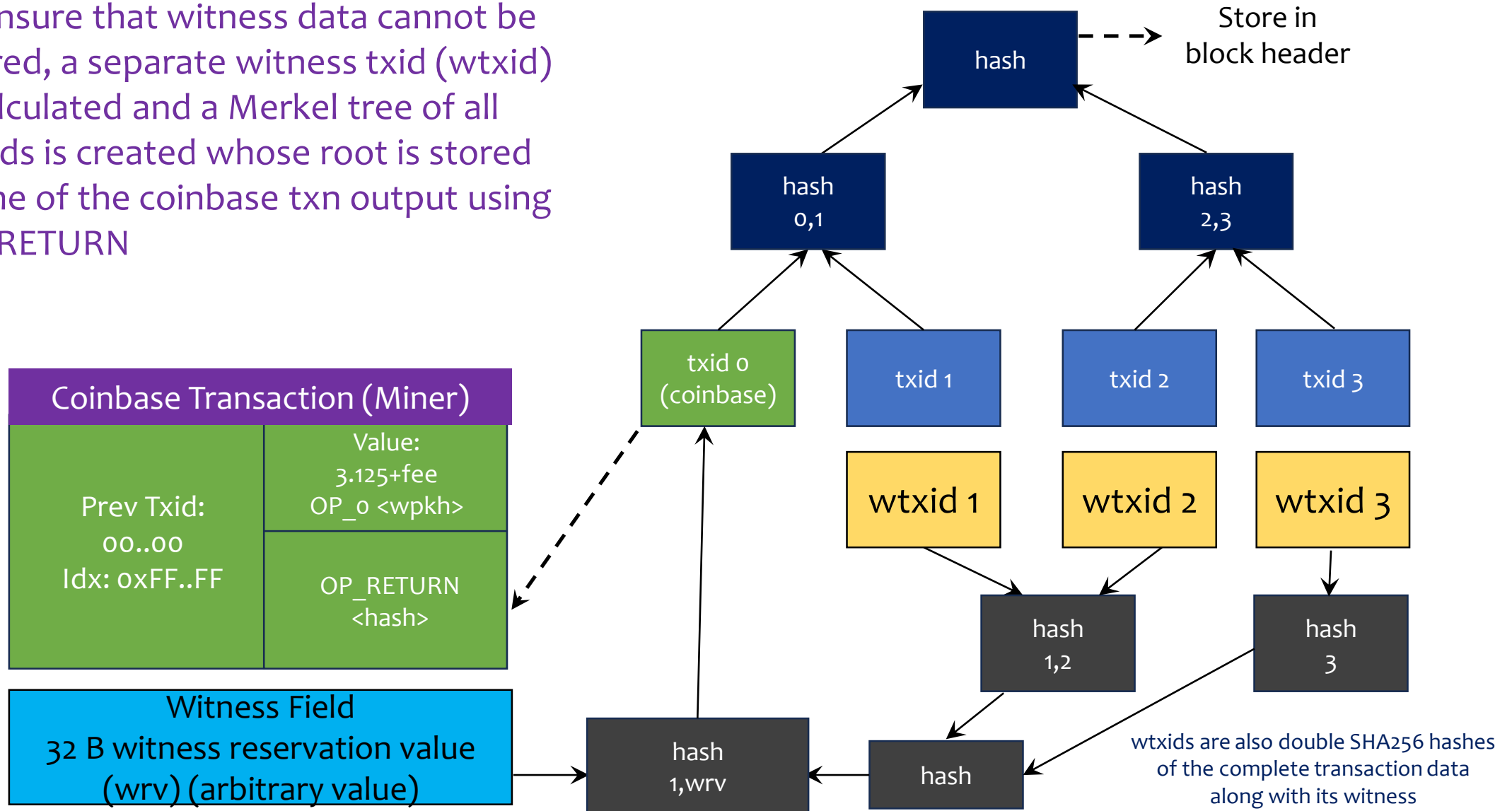
Coinbase Transaction

- It is a special txn created by the miner attempting to create a block
- It is also the first txn in a candidate block



Committing Witness Data to the Block

To ensure that witness data cannot be altered, a separate witness txid (wtxid) is calculated and a Merkle tree of all wtxids is created whose root is stored in one of the coinbase txn output using OP_RETURN



Lecture 17

Tuesday, Mar 31, 2026

Prevent Cheating in Payment Channels

- Suppose Alice broadcasts an old commitment (C1a) where she has a higher balance
 - Bob and everyone else know that Alice has published C1a
 - Bob can immediately redeem his (smaller) output but that does not stop Alice
 - Once the time lock expires, Alice still walks away with more funds than her share
- Bob could try to counter Alice by publishing his own old commitment (C1b) to make it a race
 - There is no guarantee of winning this race

Time Locks alone cannot prevent cheating

Add Punishment for Cheating

- The addition of an economic penalty that is higher than the intended reward gained from cheating is the only effective way to discourage the publication of an old state
- Suppose Alice publishes an old commitment (C1a) where she has a higher balance
 - Her output is time locked
 - If Bob could also take Alice's output immediately, then this would make publishing an old commitment truly uneconomical (Alice loses everything)
- Add a spending condition with the help of a revocation key that lets either party immediately claim the counterparty's time-locked output *if an outdated commitment is ever published*

IF (revocation path is used)

Require: Signature from the Revocation key // For Bob to punish Alice

ELSE (normal time lock path)

Require: Time lock // Alice must wait

Require: Signature from Alice Key // Alice claims her output after delay

END

Revocation Key – Naïve Construction

- To enable punishment through revocation key, let's try a straightforward key construction method
 - Bob generates a key pair (r_b , R_b)
 - *Lowercase letters denote private key, uppercase letter denotes public key*
 - He uses R_b as the revocation key in C1a, signs and gives it to Alice
 - The revocation path lets Bob take Alice's funds immediately because Bob knows private key r_b
- However, there is a serious flaw with this method
 - If Alice honestly publishes the latest commitment, she must wait to claim her funds
 - Bob already knows r_b , so he can steal everything even when Alice does nothing wrong
- We need a revocation key that
 - Can punish only if an outdated commitment is published
 - Cannot be used against the latest commitment

A revocation public key is required at the time of constructing a commitment transaction, but the corresponding revocation private key should remain hidden from both the parties until they advance to a new commitment

Safe Revocation Key

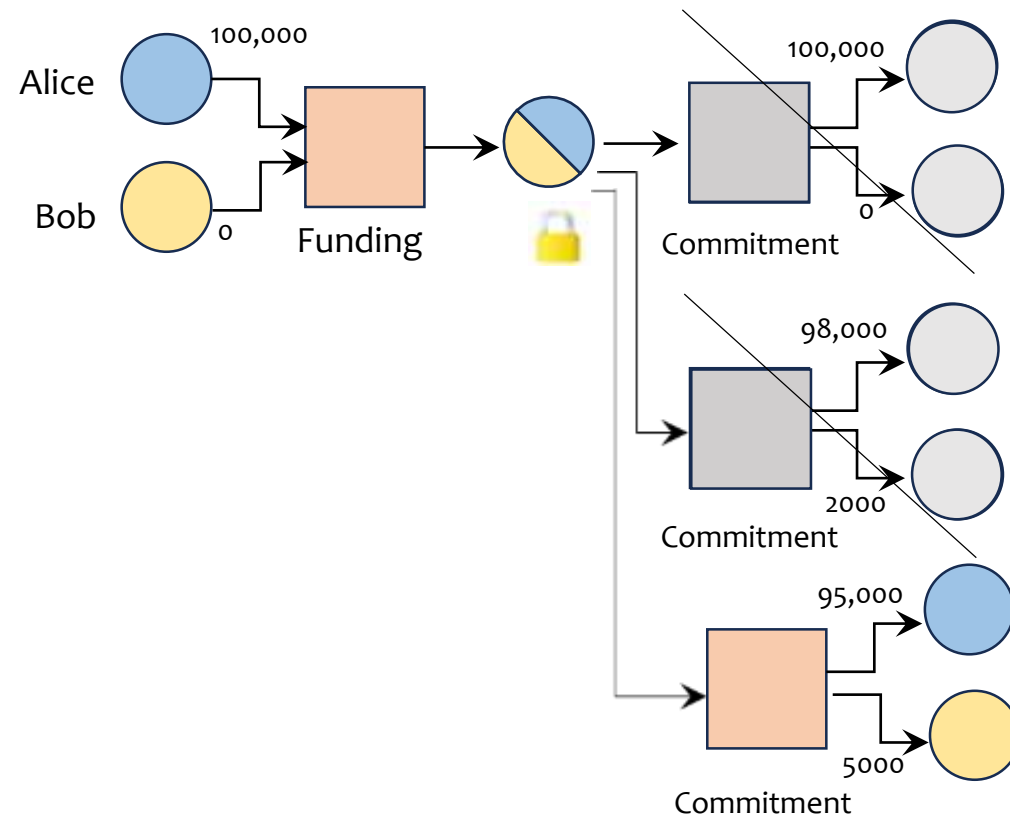
A safe revocation key enables punishment only when a commitment becomes outdated

- Each commitment transaction ($C_{1a}, C_{1b}, C_{2a}, \dots$) has its own revocation key pair (r, R)
- Let's focus on constructing a safe revocation key for C_{1a} , which Bob creates for Alice
 - Bob should be able to punish Alice only if she publishes this state after it's obsolete
 - Bob must not be able to take Alice's funds if she publishes it honestly while it's the latest state
- Alice generates a key pair (r_a, R_a)
- Bob generates a key pair (r_b, R_b)
- They exchange the public keys R_a and R_b
- Bob combines them to make a revocation public key $R = R_a + R_b$
- Bob constructs C_{1a} and adds a revocation path with the revocation public key R and gives half-signed C_{1a} to Alice
- Neither Bob nor Alice knows the private key which is $r = r_a + r_b$
- Bob cannot steal Alice's funds if she publishes C_{1a} honestly

Revoking the Old Commitment

- Suppose Alice and Bob agree to move on to a new state with updated balances
- Bob has to create a new commitment C2a, which will make C1a outdated but before that he has to revoke C1a
- Bob asks Alice to reveal r_a
- If Alice wants to continue the channel for further micropayments, she will give r_a to Bob
- Once Bob knows r_a , he gets the private key: $r = r_a + r_b$
 - Bob can easily check if the key provided by Alice is correct by generating R
- If Alice tries to publish C1a, Bob can claim her funds too
- Alice does not know r because Bob never reveals r_b
- Now, Bob is safe to move forward and constructs C2a for Alice
- Alice has no incentive to publish C1a because she will lose all her funds
- This effectively revokes C1a

The Complete Picture



Watch Tower

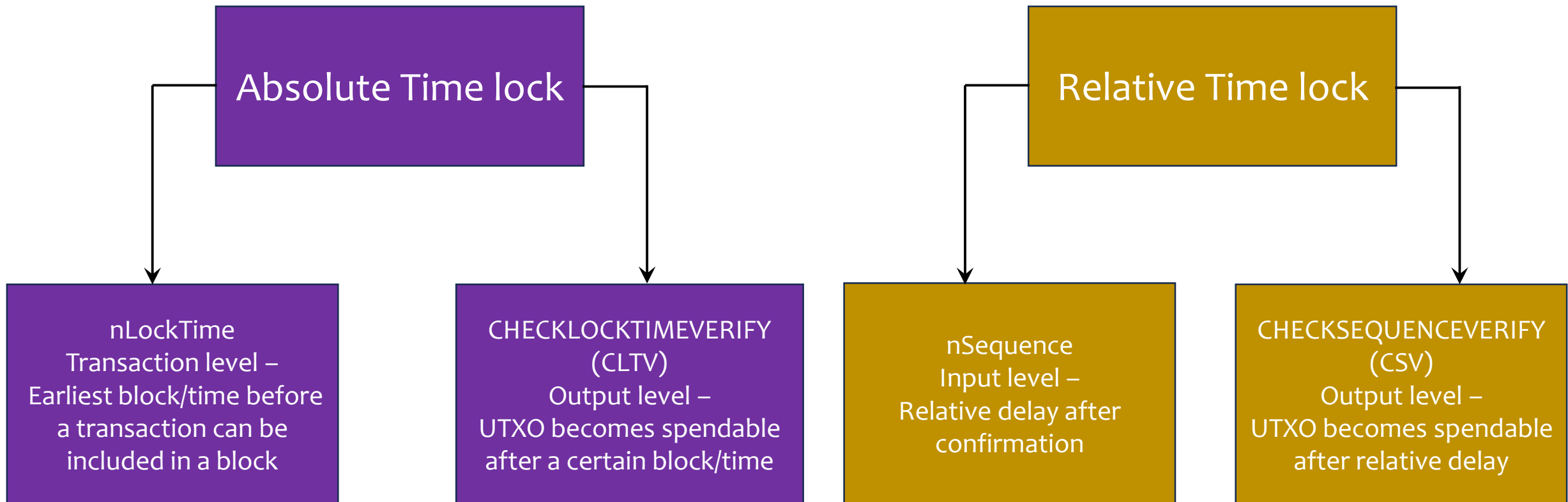
If the honest party isn't online when an outdated commitment is published, the cheater can wait out the time lock and steal the funds

Job of monitoring the blockchain can be outsourced to a dedicated service called a watchtower

With payment channels, revocation mechanism, and watchtowers in place, Bitcoin can be securely used for micropayments while avoiding the high on-chain fees for every transaction

Time locks

- Time locks are encumbrance that stop a transaction or its output from being confirmed or spent until a certain time or block height is reached
- Absolute time lock: Locks until a specific block height or time
- Relative time lock: Locks for a period of time after confirmation



Lecture 18

Thursday, Apr 2, 2026

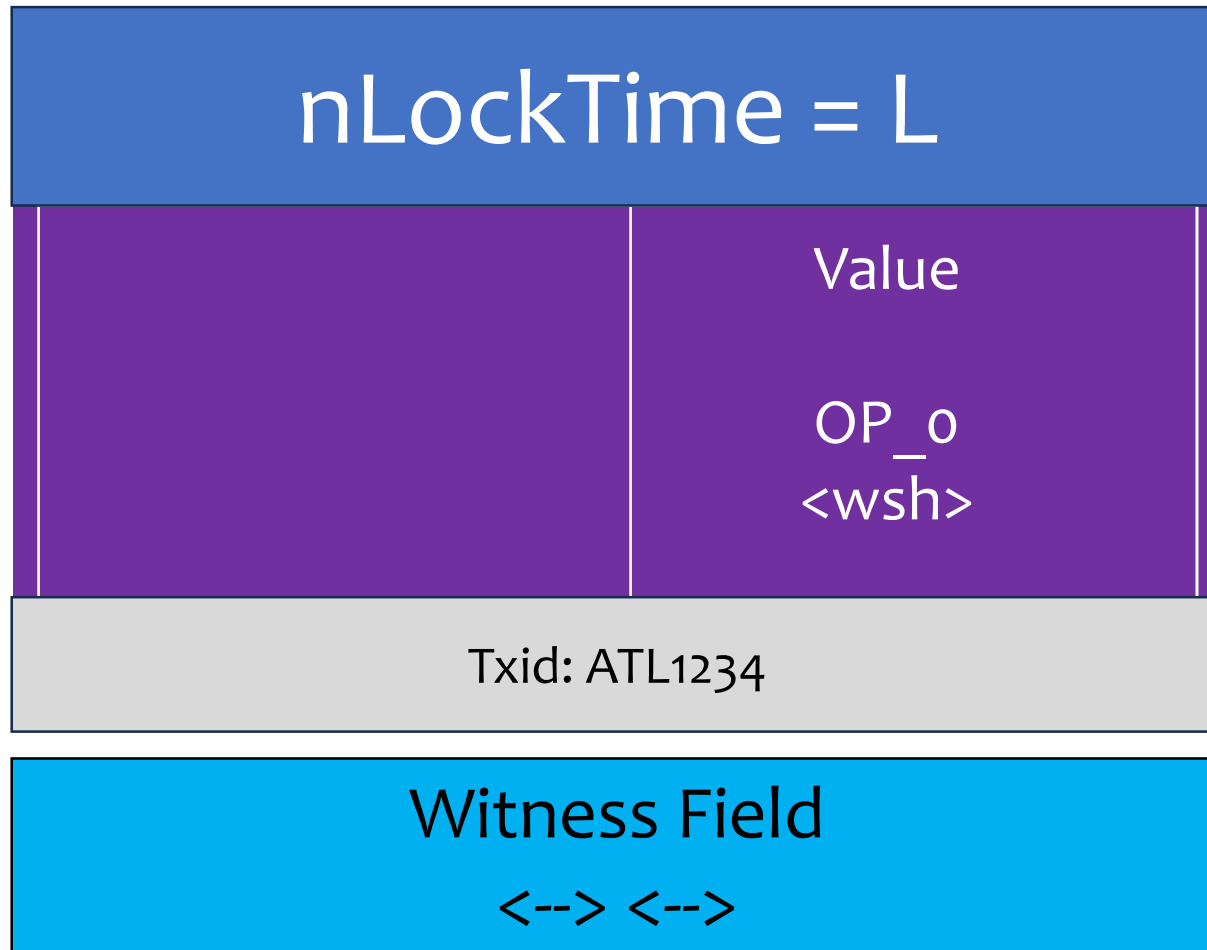
Absolute Time Lock

Absolute time lock: Locks until a specific block height or time

- If $L < 500,000,000$ – considered as a block height
- If $L \geq 500,000,000$ – considered as a Unix timestamp which is the number of seconds since Jan 1, 1970
- Example: Suppose current block height is 915,600 and we want a transaction to get into a block roughly after two week from now. What should be the value of L in block height?
 - Average block time is 10 minutes – 2016 blocks in two weeks ($6 \times 24 \times 14 = 2016$ blocks): $nLockTime = 917,616$
- Example: Suppose we want a transaction to get confirmed after April 30, 2026, 10 am. What should be the value of L in Unix timestamp?
 - Using a Unix timestamp calculator, we find that $nLockTime = 1777525200$
<https://www.unixtimestamp.com/> (Unix timestamp calculator)

Transaction Lock

Suppose current block height & time at which this transaction is created are B & T respectively

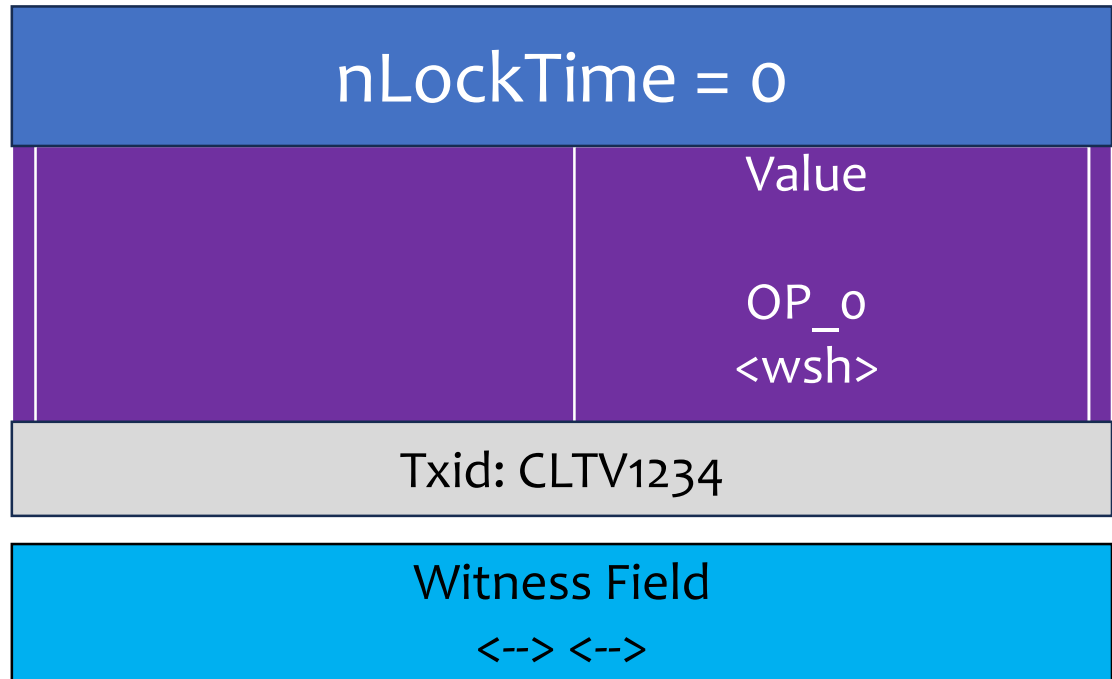


nLockTime = L : This whole transaction cannot be mined and included in a block until the absolute block height or absolute time L has reached

If $L < 500M \rightarrow$ valid when $B \geq L$

If $L \geq 500M \rightarrow$ valid when $T \geq L$

CHECKLOCKTIMEVERIFY (CLTV)



Witness Script (ws):

```
<L> OP_CHECKLOCKTIMEVERIFY  
OP_DROP  
<pubkey> OP_CHECKSIG
```

<wsh> = SHA256(ws)

CLTV Lock: The transaction **can be immediately mined and included in any block (B+1, B+2, ...)**. However, its output has a CLTV lock, which prevents its spending by a child transaction until the time lock condition L is met

<L> OP_CHECKLOCKTIMEVERIFY OP_DROP

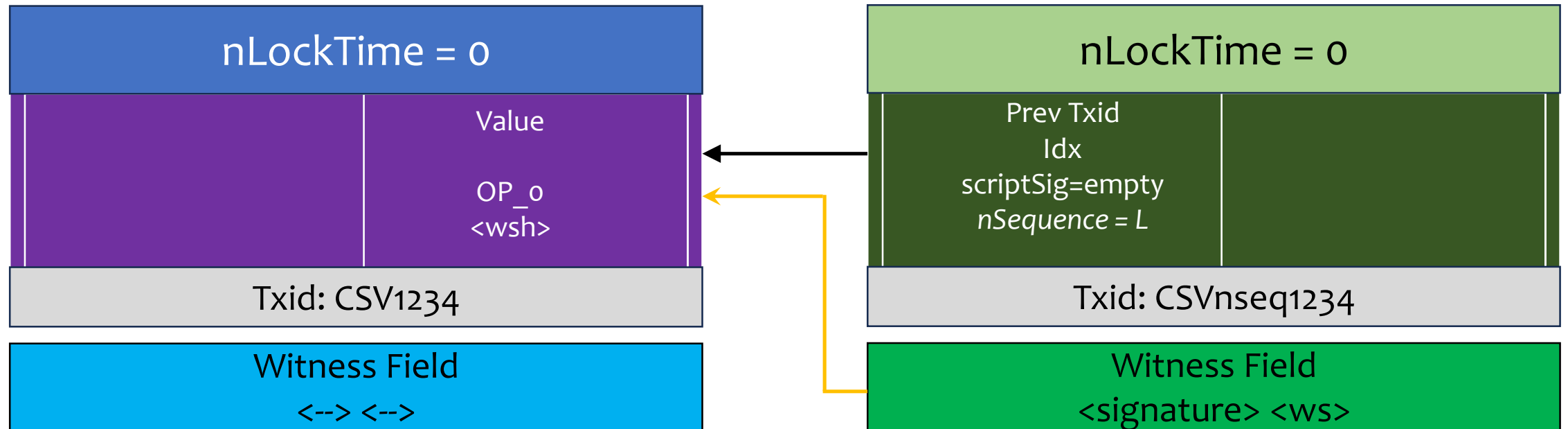
If the absolute time lock condition fails, the script fails, otherwise, the script succeeds, and OP_DROP removes <L> from the stack, allowing the execution and checking of the subsequent operations on the stack

Relative Time Lock

- Relative time locks can be specified in either blocks or time units
- A *relative time lock of L blocks* on a UTXO means that it can only be spent after at least L confirmations
- A *relative time lock of L time units* on a UTXO means that it can only be spent after at least L time units have passed since its confirmation
- A time unit is defined as 512 seconds whereas a block is generated on average after 600 s
 - The value of 512 is chosen because it is equal to 2^9 , which simplifies encoding
- A flag bit in the BIP68 encoding scheme is used to distinguish whether the relative time lock is specified in blocks or in time units

CSV + nSequence

Relative time lock is enforced through a combination of CSV script and nSequence field



- Even if the parent output's relative lock has expired, the child is still required to set `nSequence ≥ L`
- `nSequence` signals to the script that the input is spending a relative lock output to differentiate it from normal outputs without any locks

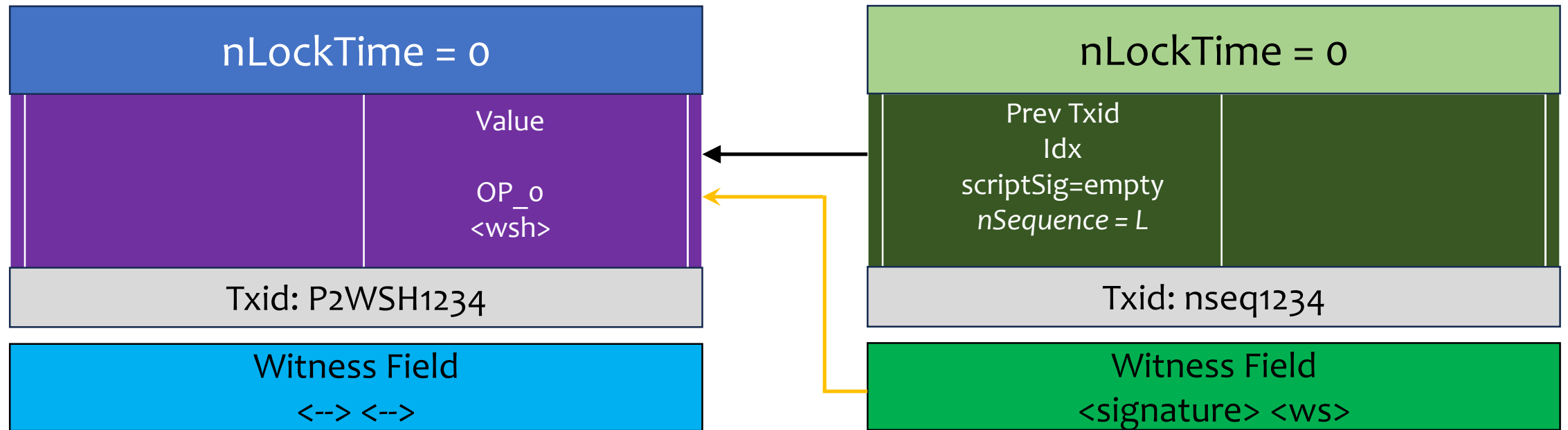
Witness Script (ws):

```
<L> OP_CHECKSEQUENCEVERIFY OP_DROP  
<pubkey> OP_CHECKSIG
```

`<wsh> = SHA256(ws)`

Relative Time Lock: nSequence Only

- Suppose a normal transaction is confirmed in block B and it has no time lock on its output
- We want to spend it after a delay of L blocks by creating a child transaction by appropriately setting its nSequence value
- What is the problem with this method and why it won't work?



- nSequence cannot enforce the lock without CSV
 - This transaction is immediately spendable because nSequence has no effect (miners ignore nSequence field without a CSV lock)

Summary of Time Locks

Block 1000 – Parent Txn mined (if applicable)

nLockTime (L=1010): Block 1000-1009 → Txn rejected
 Block 1010 → Txn mined

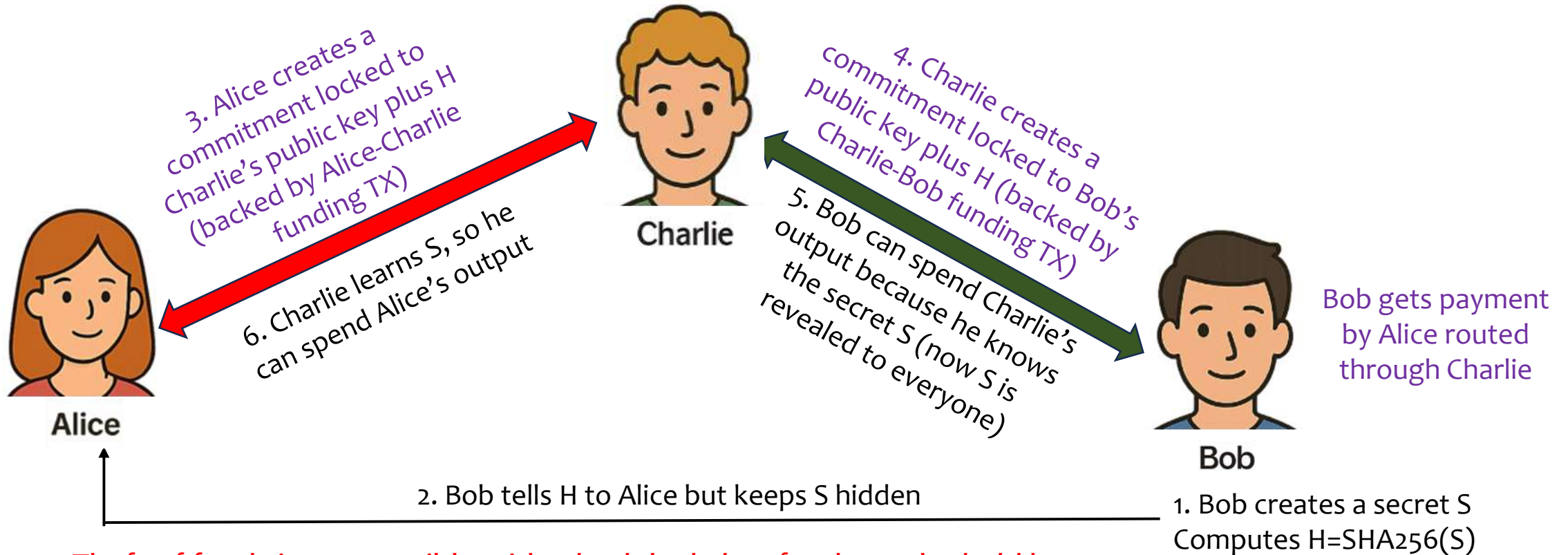
CLTV Output (L=1010): Output remains locked until 1010
 Block 1000-1009 → child Txn rejected (cannot spend)
 Block 1010 → child Txn mined (output spent)

CSV + nSequence: Output relative lock = 100 blocks
 Block 1001 → child Txn rejected (nSequence = 100)
 Block 1100 → child Txn still rejected (age check)
 Block 1101 → child Txn mined

nSequence only on normal output: Block 1001 → child Txn mined immediately
 Block 1002 → competing Txn can also spend parent

Payment Routing

- Alice wants to buy coffee from Bob and pay in BTC
 - Alice does not have a direct payment channel with Bob
- Can Alice route her payment to Bob through Charlie who is connected to both in a trustless way?



- Theft of funds is not possible with a hash lock, but funds can be held hostage
 - Bob can hold Charlie's funds hostage if he never reveals S (never claims funds so never gets paid too)
 - Charlie can hold Alice's funds hostage if he never creates a transaction for Bob

Hash Lock

- A Hash lock is a type of lock that restricts the spending of an output until the corresponding preimage of a specified hash (H) is publicly revealed
 - OP_SHA256 <H> OP_EQUALVERIFY
 - Once any hash lock is opened publicly, any other hash lock secured using the same preimage can also be opened
 - This makes it possible to create multiple outputs that are all encumbered by the same hash lock and which all become spendable at the same time

Hash locks allow payments to be routed through third parties without any risk that the third-parties will take the payment for themselves

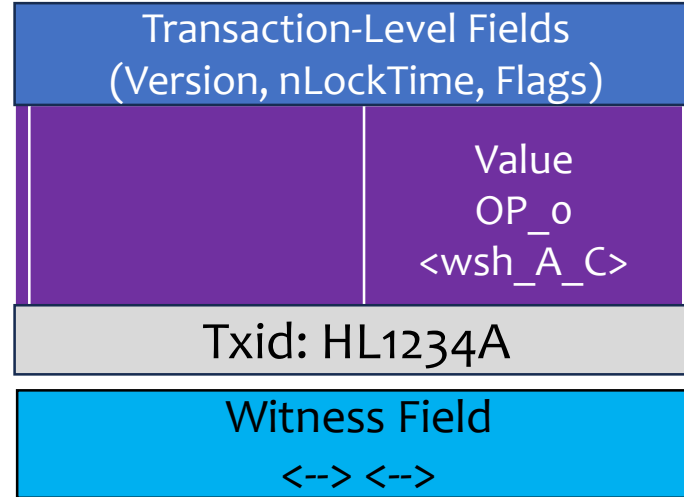
Hash Lock Transactions & Scripts

Witness Script (ws_A_C):

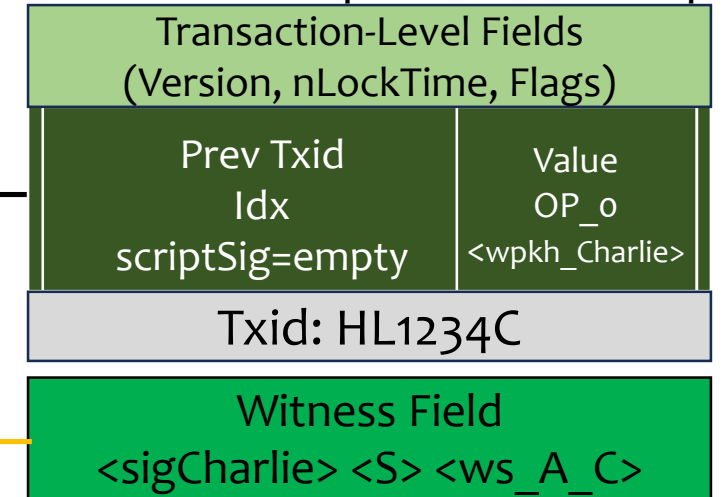
```
OP_SHA256 <H> OP_EQUALVERIFY  
<PubKeyCharlie> OP_CHECKSIG
```

$\text{<wsh_A_C> = SHA256(ws_A_C)}$

Alice gives this txn to Charlie



Charlie learns S and spends Alice's output

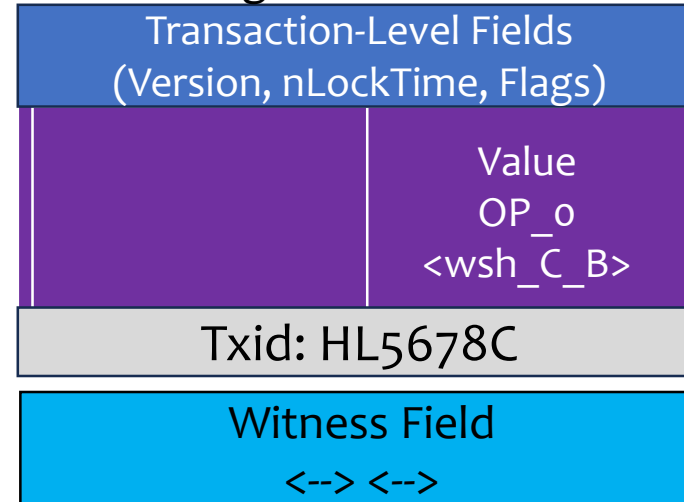


Witness Script (ws_C_B):

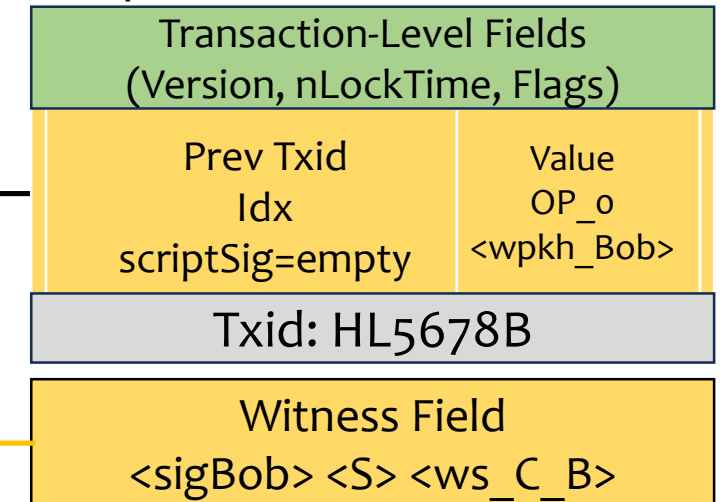
```
OP_SHA256 <H> OP_EQUALVERIFY  
<PubKeyBob> OP_CHECKSIG
```

$\text{<wsh_C_B> = SHA256(ws_C_B)}$

Charlie gives this txn to Bob



Bob spends and reveals the secret S



Hash Time Lock Contract (HTLC)

- HTLC solves the funds hostage problem by creating a refund path with the help of time lock
- Along with creating a hash lock spending condition for Charlie, Alice also creates a CSV locked spending condition for herself
 - Charlie can immediately spend the output with the help of the secret S if Bob reveals it
 - If Charlie does not cooperate or if Bob never reveals the secret S, Alice can reclaim her funds after an agreed upon relative time delay enforced through the CSV lock

Witness Script (ws_HTLC_A_C):

OP_IF

Hashlock branch: Charlie needs preimage S of H

OP_SHA256 <H> OP_EQUALVERIFY

<PubKeyCharlie> OP_CHECKSIG

OP_ELSE

Time lock branch: Alice refunds after L blocks

<L> OP_CHECKSEQUENCEVERIFY OP_DROP

<PubKeyAlice> OP_CHECKSIG

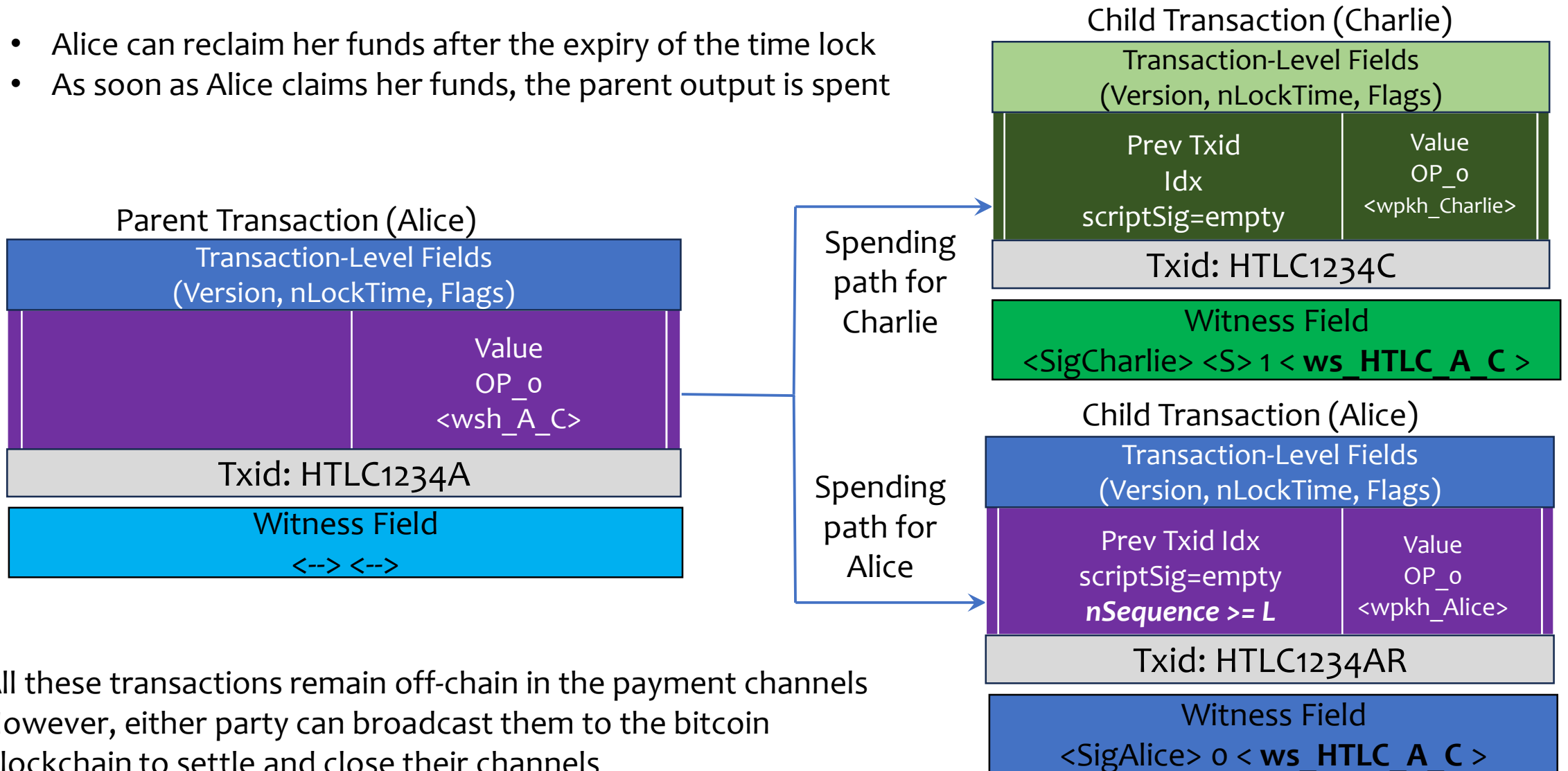
OP_ENDIF

<wsh> = SHA256(ws_HTLC_A_C)

- Charlie can spend via the hash lock path:
 - scriptSig = empty
 - Witness = <SigCharlie> <S> 1 < ws_HTLC_A_C >
 - Condition: SHA256(S) = H and Charlie's signature are valid
- Alice can refund via time lock path:
 - scriptSig = empty
 - Witness: <SigAlice> 0 < ws_HTLC_A_C >
 - Condition: L blocks (CSV) after the confirmation of the funding transaction by Alice & Alice's signature

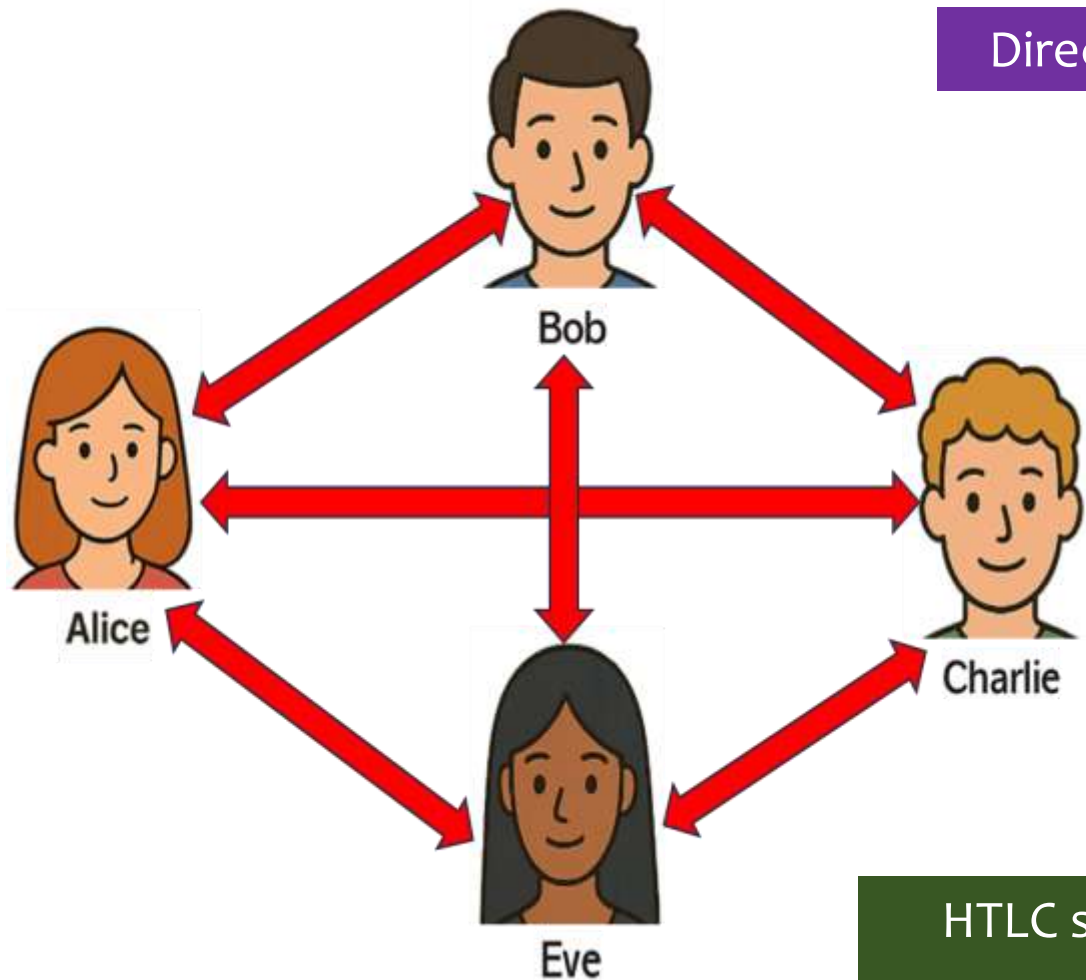
HTLC – Spending Paths

- Alice can reclaim her funds after the expiry of the time lock
- As soon as Alice claims her funds, the parent output is spent



- All these transactions remain off-chain in the payment channels
- However, either party can broadcast them to the bitcoin blockchain to settle and close their channels

Scalability Issues in Payment Channels



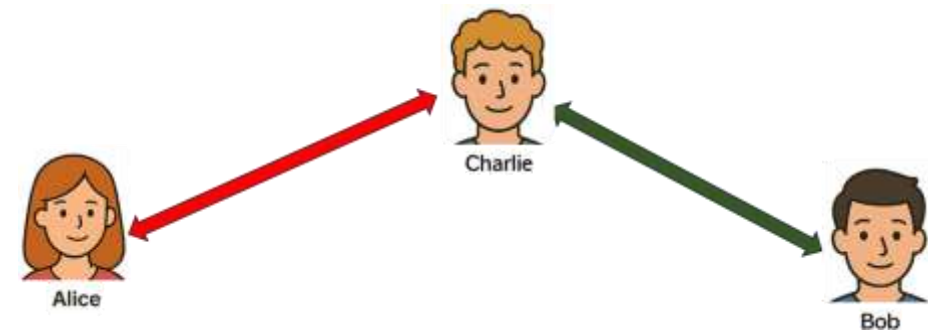
Direct payment channel between all the peers is not scalable

- On-chain funding transaction with everyone
- Channel management & monitoring overheads
- Locked funds in every channel (liquidity problems)

HTLC solves the scalability problem because it ensures secure payment routing

Micropayments with Routing

- Bob (cafe owner) creates and displays a QR code that contains the required payment and the routing information including the hash of the secret to facilitate routing
- Alice scans the QR code, creates an HTLC in her payment channel with Charlie (assuming the payment is routed through Charlie)
- In the background, Charlie creates another HTLC in his payment channel with Bob
- Bob receives the payment from Charlie, and consolidates his balance in his commitment transactions with Charlie
- Charlie receives the payment from Alice, and consolidates his balance in his commitment transactions with Alice
- Payment received message is displayed in Bob's wallet, payment debited in Alice's wallet
- Bob gives coffee to Alice
- Routing nodes charge very small fees (1 to 2 sats as base fee and some component proportional to txn value)



Commitment Transaction with HTLC Output

Transaction-Level Fields (Version, nLockTime, Flags)	
Funding TX	Alice's output (2 spending paths) - Alice can spend after time lock - Charlie can immediately spend Alice's funds with revocation key
	Charlie's output (1 spending path) Charlie can immediately spend his funds
	HTLC output (3 spending paths) - Anyone can claim with preimage of H - Alice can reclaim after time lock - Charlie can immediately spend with revocation key
Txid: C1a (Alice-Charlie-HTLC)	
Witness Field <Placeholder for Alice Sig> <SigCharlie><2-of-2 redeemScript>	

Alice's output script (output_alice_script)
OP_IF # Revocation path for Charlie if Alice cheats
 <revocationpubkey>
OP_ELSE # Normal delayed spend path
 <to_self_delay> OP_CHECKSEQUENCEVERIFY OP_DROP
 <pubkeyAlice> # Alice's delayed spend key
OP_ENDIF
OP_CHECKSIG

<wsh_output_1>=SHA256(output_alice_script)

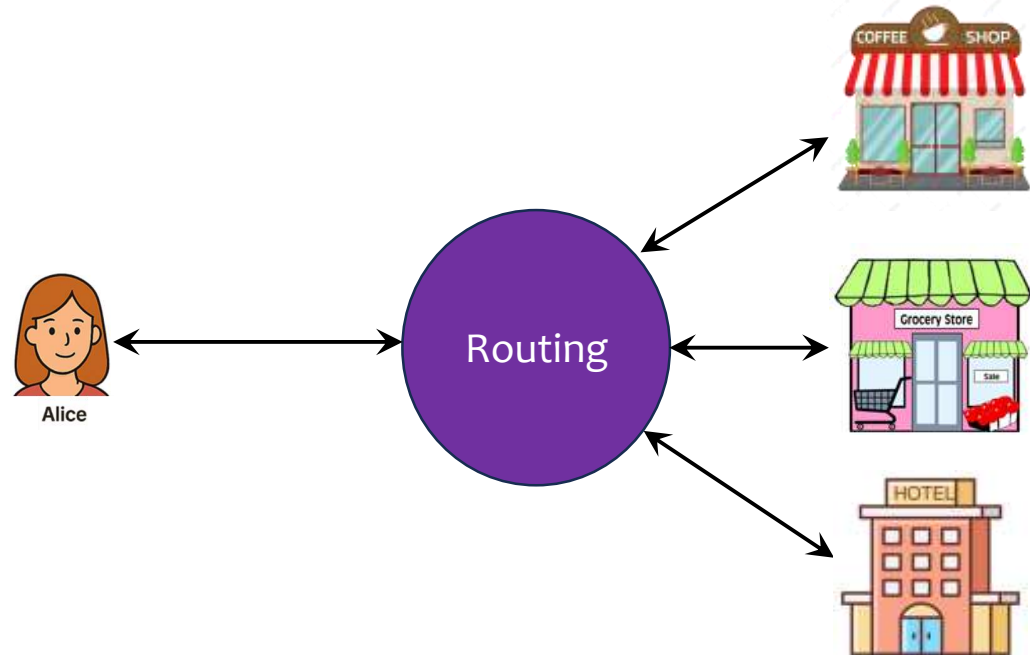
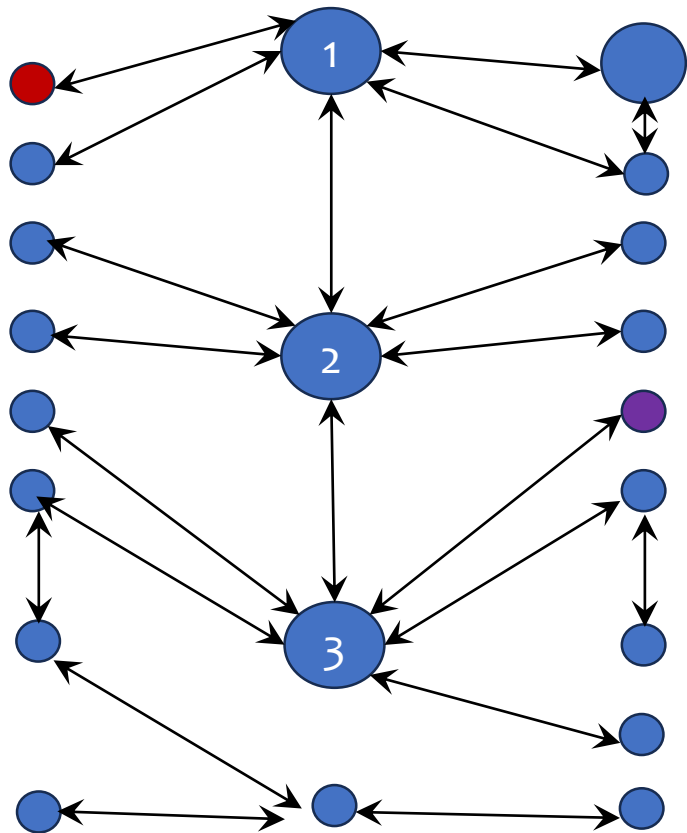
Charlie's output has no script
OP_0 <p2wpkh_Charlie>

HTLC output script (output_htlc_script)
OP_IF # Revocation path for Charlie if Alice cheats
 <revocationpubkey>
OP_ELSE
 OP_IF # Anyone can claim HTLC with preimage of H before timeout
 OP_SHA256 <H> OP_EQUALVERIFY
 <recoverpubkey>
 OP_ELSE # Alice refunds after CLTV timeout
 <cltv_expiry> OP_CHECKLOCKTIMEVERIFY OP_DROP
 <pubkeyAlice>
 OP_ENDIF

OP_ENDIF
OP_CHECKSIG
<wsh_output_htlc>=SHA256(output_htlc_script)

Network of Payment Channels

- A network of payment channels emerges with the help of trustless routing enforced through HTLC
- Each channel becomes an edge in a graph and payment can be safely routed across multiple hops
- Typical pathfinding algorithms like Dijkstra algorithms can be used to find the optimal paths (shortest path or with lowest fees)



Some nodes (1, 2, 3 in the graph) become highly connected hubs, offering routing as a service and earning small routing fee

Lightning Network

- The concept of payment channels for off-chain Bitcoin transactions was introduced in 2015 in a Whitepaper by Joseph Poon & Thaddeus Dryja
 - “The Bitcoin Lightning Network: Scalable On-Chain Instant Payments”
- Lightning network (LN) was launched by Lightning Labs in 2018
 - After the activation of SegWit on the Bitcoin blockchain that solved the transaction malleability problem
 - The ideas that we discussed are at the core principles of the LN
- LN is called a layer 2 protocol for Bitcoin
 - It is neither a blockchain nor a cryptocurrency
- LN continues to grow and adapt

Number of Nodes	Number of Channels	Network Capacity	Average Node Age	Average Channel Age	Average Channels per Node
12,599 ↑+1.70%	43,565 ↑+0.9%	3,868.83 BTC ↓-1% \$478,641,308.57	1,104.8 days 3 years	645.8 days a year	6.92

The LN is a critical component of Bitcoin scalability, and many ideas for Bitcoin's future evolution revolve around it

Privacy Evolution in Bitcoin

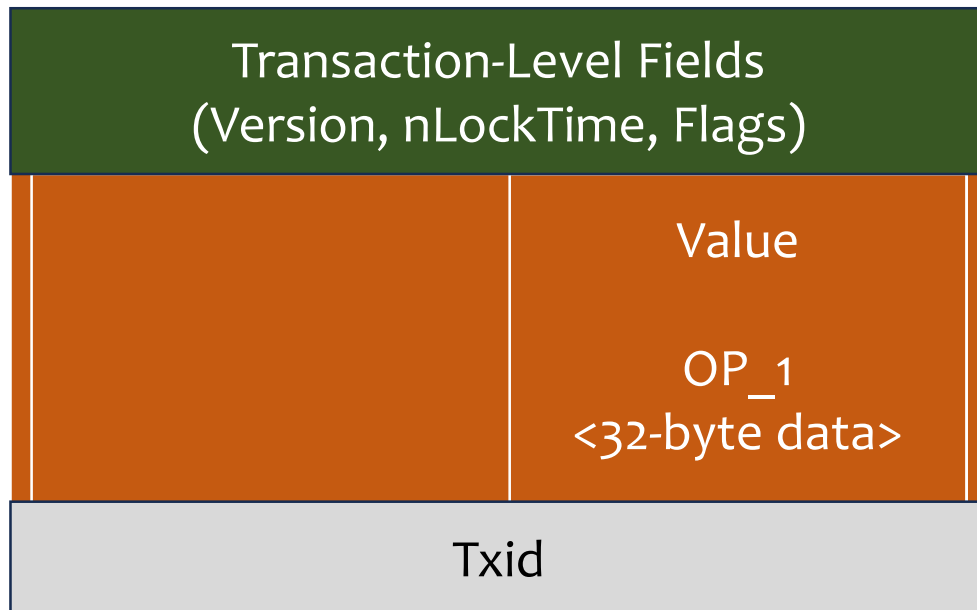
Privacy Evolution in Bitcoin

- Privacy: The goal is to hide how you are using your coins; it is not about hiding your identity or making you anonymous
 - In banking, your transactions are hidden from public view and only the bank knows how you are spending your money
 - Bitcoin is a public ledger with all the transactions available on-chain
- How to hide the spending scripts in Bitcoin?
- Legacy transactions (Very poor privacy)
 - The outputs reveal a lot of information including the pubkeyhash, multisig public keys, spending paths, complexity of the lock, full script structure
- Segwit transactions (Some privacy improvements)
 - Standardized the output formats and reduced them to only two types
 - Outputs still reveal the transaction type (p2wpkh, or p2wsh) but the spending paths, script structure remain hidden initially
 - Spending conditions are revealed in the child transaction when the coins are spent and once again the entire script structure becomes visible
- Can we further improve privacy?

Privacy Improvements

- Make all the outputs same i.e., only one type
- Do not reveal the entire script or all the spending paths when unlocking the coins

Only one type of output

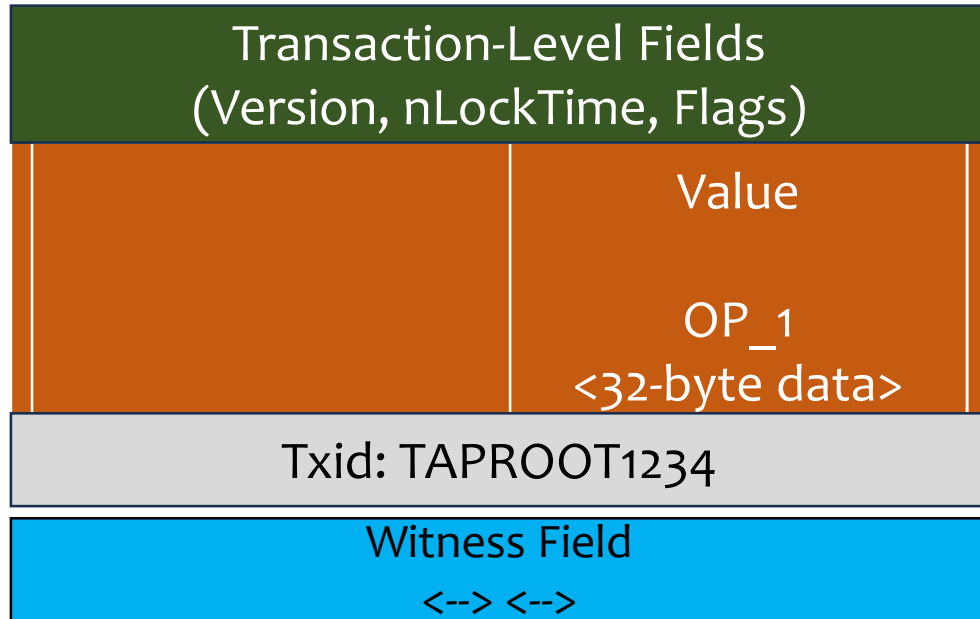


Reveal only one path when spending the output

```
HTLC output script (output_htlc_script)
OP_IF # Revocation path for Charlie if Alice cheats
  <revocationpubkey>
OP_ELSE
  OP_IF # Anyone can claim HTLC with preimage of H before timeout
    OP_SHA256 <H> OP_EQUALVERIFY
    <recoverpubkey>
  OP_ELSE # Alice refunds after CLTV timeout
    <cltv_expiry> OP_CHECKLOCKTIMEVERIFY OP_DROP
    <pubkeyAlice>
  OP_ENDIF
OP_ENDIF
OP_CHECKSIG
<wsh_output_htlc>=SHA256(output_htlc_script)
```

Bitcoin supports both these privacy improvements through the ‘Taproot upgrade’

Taproot Upgrade



Technical improvements:

- Signature: Replaced ECDSA with Schnor Signature (only one output structure, batch verification, privacy benefits)
- MAST: Merkelized Abstract Syntax Tree (only reveal the script branch that is used for spending, while keep all other options hidden)
- Tapscript: New op-code, new interpretation of some op-codes to validate Schnor and MAST

- Taproot is the second major Bitcoin upgrade
 - Activated in Nov 2021 at the block height 709,632, as a soft fork
- It has only one output type for all type of spending conditions called pay-to-taproot (P2TR)
 - OP_1 <32-byte data>
 - OP_1 is an indication that it is a taproot transaction
- Legacy as well as SegWit nodes accept taproot transactions
 - Legacy nodes treat them as “anyone can spend”
 - SegWit V0 nodes know it is a future version with OP_1 so they accept them
- The witness field in the child transaction contains the required data that allows to unlock the taproot output
 - While unlocking, only one spending path is revealed, and all the other branches in the script remain hidden

Privacy Advantages of Schnorr Signature

Hides the mutlisig public keys through key aggregation

- Suppose Alice, Bob and Charlie want to create a 3-of-3 multisig
 - They first aggregate their public keys $\langle pkAgg \rangle = \langle pkA \rangle + \langle pkB \rangle + \langle PkC \rangle$
 - They then derive a 32 B tweaked public key Q from $\langle pkAgg \rangle$
 - Lock funds in a P2TR transaction output: $OP_1 \langle Q \rangle$
- To unlock, they cooperate to create a spending transaction because they only know their individual private keys
 - Each party individually signs the spending transaction with the help of its private key and produce partial signatures $\langle sigA \rangle$, $\langle sigB \rangle$ and $\langle sigC \rangle$
 - They aggregate the signatures: $\langle sigAgg \rangle = \langle sigA \rangle + \langle sigB \rangle + \langle sigC \rangle$
 - The witness field of the spending transaction only contains $\langle sigAgg \rangle$ (*transaction size is also reduced so lower fee becomes an added advantage*)

The spending transaction now reveals nothing about the number of participants or their individual public keys (great for privacy)

Batch Verification

- Key aggregation and signature aggregation are powerful features of Schnorr signatures that also accelerate transaction verification
- In a single multisig transaction, multiple public keys and signatures can be aggregated into one, improving efficiency and privacy
- At the block level, validators can use batch verification, where multiple Schnorr signatures from independent transactions are combined into a single verification equation
- If this batch check succeeds, all signatures are validated simultaneously
- If it fails, the validator falls back to individual verification to identify the invalid signature

Merkelized Abstract Syntax Tree (MAST)

Only reveal the script branch that is actually used for spending, while keeping all other spending conditions hidden

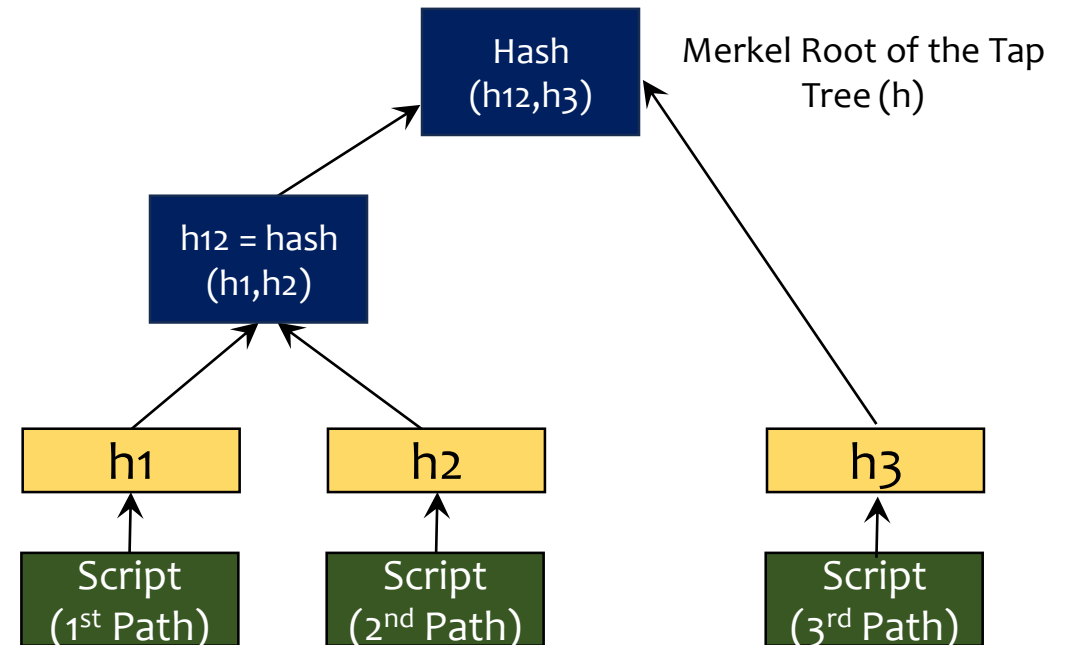
```
HTLC output script (output_htlc_script)
OP_IF # Revocation path for Charlie if Alice cheats
  <revocationpubkey>
OP_ELSE
  OP_IF # Anyone can claim HTLC with preimage of H
    before timeout
    OP_SHA256 <H> OP_EQUALVERIFY
    <recoverpubkey>
  OP_ELSE # Alice refunds after CLTV timeout
    <cltv_expiry> OP_CHECKLOCKTIMEVERIFY
  OP_DROP
    <pubkeyAlice>
  OP_ENDIF
OP_ENDIF
OP_CHECKSIG
<wsh_output_htlc>=SHA256(output_htlc_script)
```

3 spending conditions – 3 leafs in the MAST

Leaf 1 (h1): Script for the first path (revocation key)

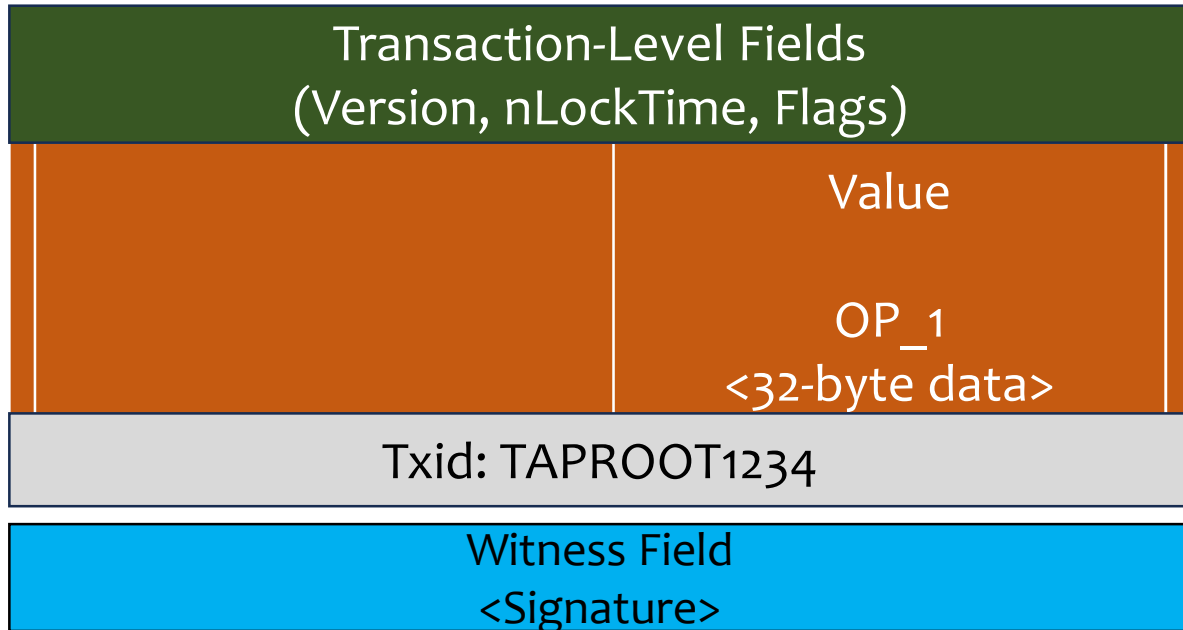
Leaf 2 (h2): Script for the second path (HTLC path)

Leaf 3 (h3): Script for the third path (Refund path)



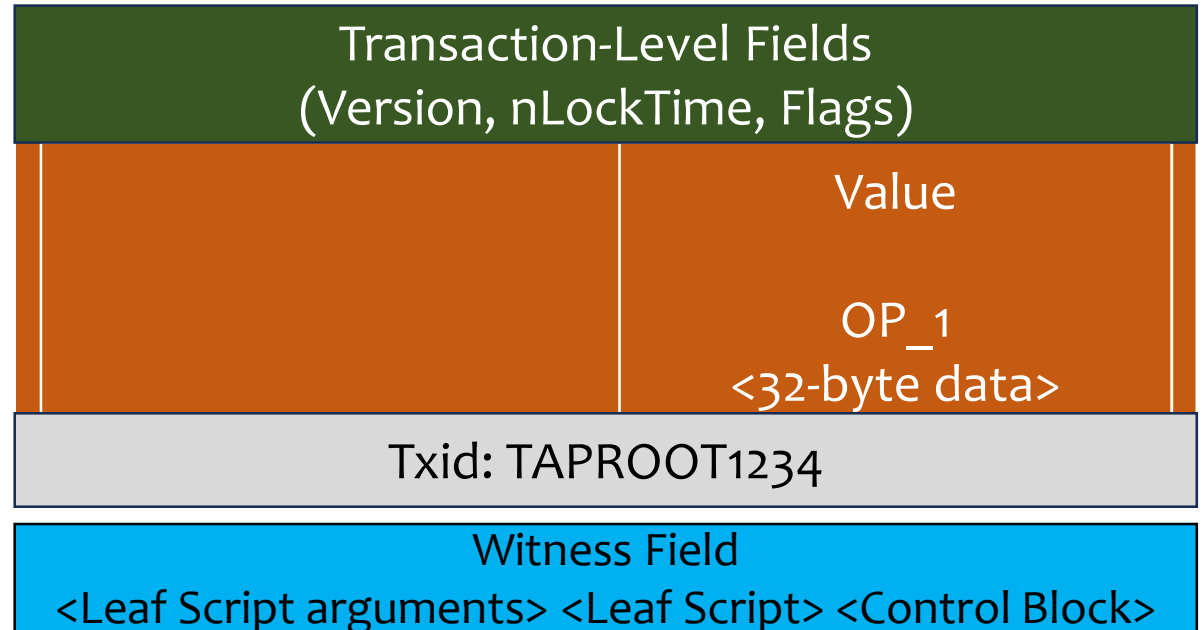
Witness Elements

Spending from Key path



Only one witness element, which is the signature created with the tweaked private key

Spending from Script path



The script of the spending path only along with the arguments needed by the script (such as signature, hash, something else)

The control block proves that the included script is part of the committed taproot key in the tweaked public key (provides the Merkel path hashes)

Public Key Privacy

- Public keys are sensitive privacy identifiers
- Bitcoin design has evolved to minimize the on-chain visibility of raw public keys
 - In P2WPKH & P2WSH, raw public keys are exposed in the spending transactions (single user public key in p2wpkh, and multisig public keys in p2wsh)
 - Taproot improves privacy by locking funds to a tweaked public key and in the spending transaction only the signature is required
 - In taproot multisig transactions, key aggregation and signature aggregation techniques, which are the key features of Schnorr signature conceal the public keys (multisig looks like a single sig transaction)
 - The actual public key is only exposed when spending through the script path in taproot
- A growing best practice is to generate a new key pair for every transaction
 - Exposing public key only to the counterparty and never broadcasting it unnecessarily

Scope & Limits of Privacy in Bitcoin

- **Wallet level**

- Some privacy features can be implemented entirely within the wallet software, without requiring any change to Bitcoin's consensus rules

- **Protocol level**

- Some privacy features require a consensus-level change, usually via a soft fork, to introduce new transaction rules, opcodes, or validation logic

- There are three dimensions of transaction privacy

- **Sender privacy**

- Solutions exist to achieve sender privacy entirely at the wallet level (e.g., PayJoin protocol)

- **Receiver privacy**

- Silent or Stealth Payments (wallet level): Sender derives a one-time address from the receiver's public key using cryptography
- Receiver continuously scans the blockchain to detect payments but only they can unlock them, breaking the payer–payee link

- **Value privacy**

- Conceals transaction amounts using cryptographic proofs (solutions exist)
- Requires consensus-level changes and likely a hard fork – Not currently possible on Bitcoin